

MERN – Développeur React Full Stack

Dr. Tarek BCHINI

Objectifs

- Développer un site web complet Full Stack de vente en ligne avec React
- Le site est léger grâce aux avantages de React permettent le rafraichissement d'un seul composant en cas d'une action de rafraichissement de page web
- Apprendre à développer le Frontend et le Backend/BD
- Apprendre à sécuriser le site et l'authentification des utilisateurs
- Réalisation des interfaces en Front et la gestion en Back
- Implémentation du paiement en ligne avec PayPal
- Déploiement de la solution en ligne

Prérequis, besoin matériel et nature de la formation

+ Prérequis :

- Connaitre le minimum de la programmation et l'algorithmie
- Connaitre le minimum des commandes Linux
- Connaitre le minimum sur le fonctionnement web et les bases de données
- Connaitre le minimum des langages : html & css, javascript, bootstrap
- Avoir un compte sur Github

+ Besoin matériel :

- PC Windows 10 minimum 4G de RAM

+ Nature de la formation :

- 10% théorique et 90% pratique (projets réels)
- Manipulation et exécution de codes et commandes sur place

Plan

Niveau 1-1 : Débutant basique (24 heures)

- + Environnement et outils nécessaires
- + Frontend
 - Démarrage première application React
 - Adapter l'application pour Github
 - Créer le Header et le Footer
 - Liste des produits – statique
 - Implémenter le React Router pour les liens
 - Afficher les étoiles pour un produit
 - Afficher les détails d'un seul produit

Plan

Niveau 1-2 : Débutant confirmé (40 heures)

+ Backend

- Fonctionnement en Full Stack
- Créer le Backend et initialiser les routes

+ Frontend et Backend

- Faire communiquer le Front et le Back
- Commencer à travailler avec MongoDB
- Connecter la BD avec l'application
- Préparation des données de la BD
- Utilisation de Postman pour le Backend
- Utilisation de Redux

Plan

Niveau 2 : Intermédiaire (48 heures)

+ Réalisation du Frontend

- Phase de commande des produits
- Authentification des utilisateurs
- Processus de paiement

Plan

Niveau 3 : Confirmé (32 heures)

+ Gestion du Backend

- Profil de l'utilisateur
- Fonctionnalités de l'admin

Plan

Niveau 4 : Expert (32 heures)

- + Finalisation des front/back et déploiement
 - Les reviews des produits
 - Pagination et recherche des produits
 - Carrousselle des produits
 - Déploiement de la solution

Niveau 1-1 : Débutant basique

Environnement et outils nécessaires

Et

Frontend

Environnement et outils nécessaires

- Environnement
 - Windows 10, 64 bits, minimum 4G de RAM
- Outils nécessaires :
 - Navigateur Google Chrome
 - Visual Studio : éditeur de code
 - Node JS : cœur du compilateur et exécuter du JS
 - Git Bash : pour lancer des commandes Linux et traiter avec Github
 - Postman : pour gérer les requêtes back vers BD
 - MongoDB : pour gérer la base de données Mongo

Environnement et outils nécessaires

- Extensions à ajouter Visual Studio :
 - ES7 React/Redux : génération automatique de composants
 - JavaScript (ES6) code snippets : génération automatique de fonctions
 - Prettier code formatter : formatage après enregistrement
 - Auto rename tag : renommer automatiquement les tags
- Extensions à ajouter à Google Chrome
 - React Developer tools
 - Redux devtools

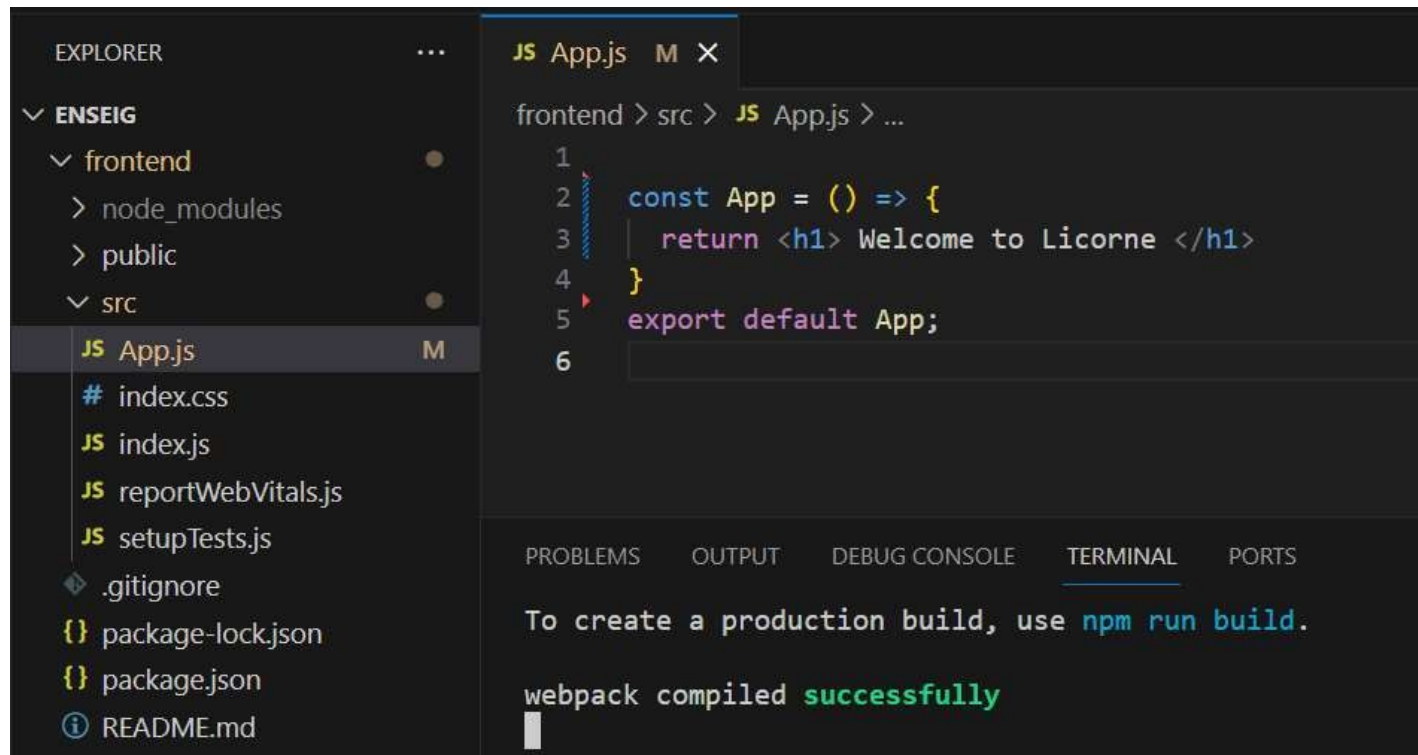
Démarrage première application React

- Créer un dossier nommé : exemple
- Ouvrir VS code et ouvrir le dossier exemple
- Ouvrir le terminal dans VS code
- Pour créer la partie frontend, dans le terminal taper :
`npx create-react-app frontend`
- Pour lancer l'application frontend, taper dans le terminal :
`cd frontend` puis `npm start`
et la première application de React sera lancée sur le navigateur web

Démarrage première application React

- On supprime les fichiers indésirables : App.css, App.test.js, logo.svg
- On supprime le contenu de App.js
- On tape rfce dans le contenu de App.js et il va nous générer une fonction
- Et faire comme suit
 - dans App.js
 - dans index.html
- Et exécuter l'application, ca va nous afficher un texte de bienvenue

app.js



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar displays the project structure under the name 'ENSEIG'. The 'src' directory is expanded, showing 'App.js' as the active file. The main editor area shows the content of 'App.js', which is a React component. The code defines a function 'App' that returns a JSX element with a heading 'Welcome to Licorne'. The component is then exported as the default export. The bottom panel shows the 'TERMINAL' tab with a message from Webpack: 'webpack compiled successfully'.

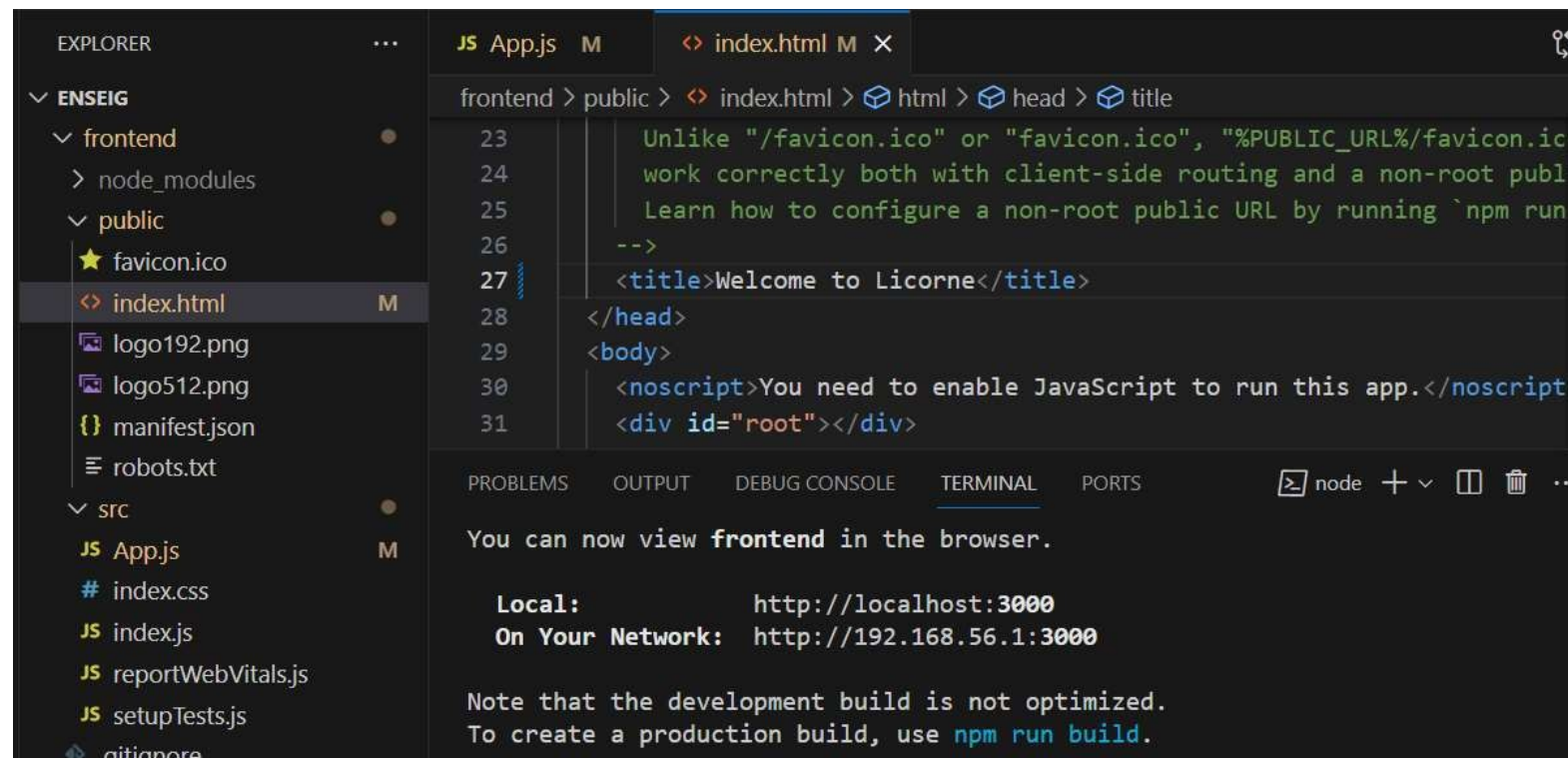
```
JS App.js M X
frontend > src > JS App.js > ...
1
2  const App = () => {
3    |   return <h1> Welcome to Licorne </h1>
4    | }
5    export default App;
6
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

To create a production build, use `npm run build`.

webpack compiled **successfully**

index.html



The screenshot shows the Visual Studio Code editor interface. On the left, the Explorer sidebar displays the project structure under the 'ENSEIG' folder. The 'public' folder is expanded, showing files like 'favicon.ico', 'index.html' (selected), 'logo192.png', 'logo512.png', 'manifest.json', and 'robots.txt'. The 'src' folder is also visible, containing 'App.js', 'index.css', 'index.js', 'reportWebVitals.js', 'setupTests.js', and '.gitignore'. The main editor area shows the 'index.html' file with the following content:

```
23 Unlike "/favicon.ico" or "favicon.ico", "%PUBLIC_URL%/favicon.ico
24 work correctly both with client-side routing and a non-root publi
25 Learn how to configure a non-root public URL by running `npm run
26 -->
27 <title>Welcome to Licorne</title>
28 </head>
29 <body>
30 <noscript>You need to enable JavaScript to run this app.</noscript>
31 <div id="root"></div>
```

Below the editor, the TERMINAL panel is active, displaying the following output:

```
You can now view frontend in the browser.

Local:      http://localhost:3000
On Your Network:  http://192.168.56.1:3000

Note that the development build is not optimized.
To create a production build, use npm run build.
```

Démarrage première application React

- index.html (dans public) contient root, appelé depuis index.js (dans src) qui appelle à son tour le app.js (dans src)

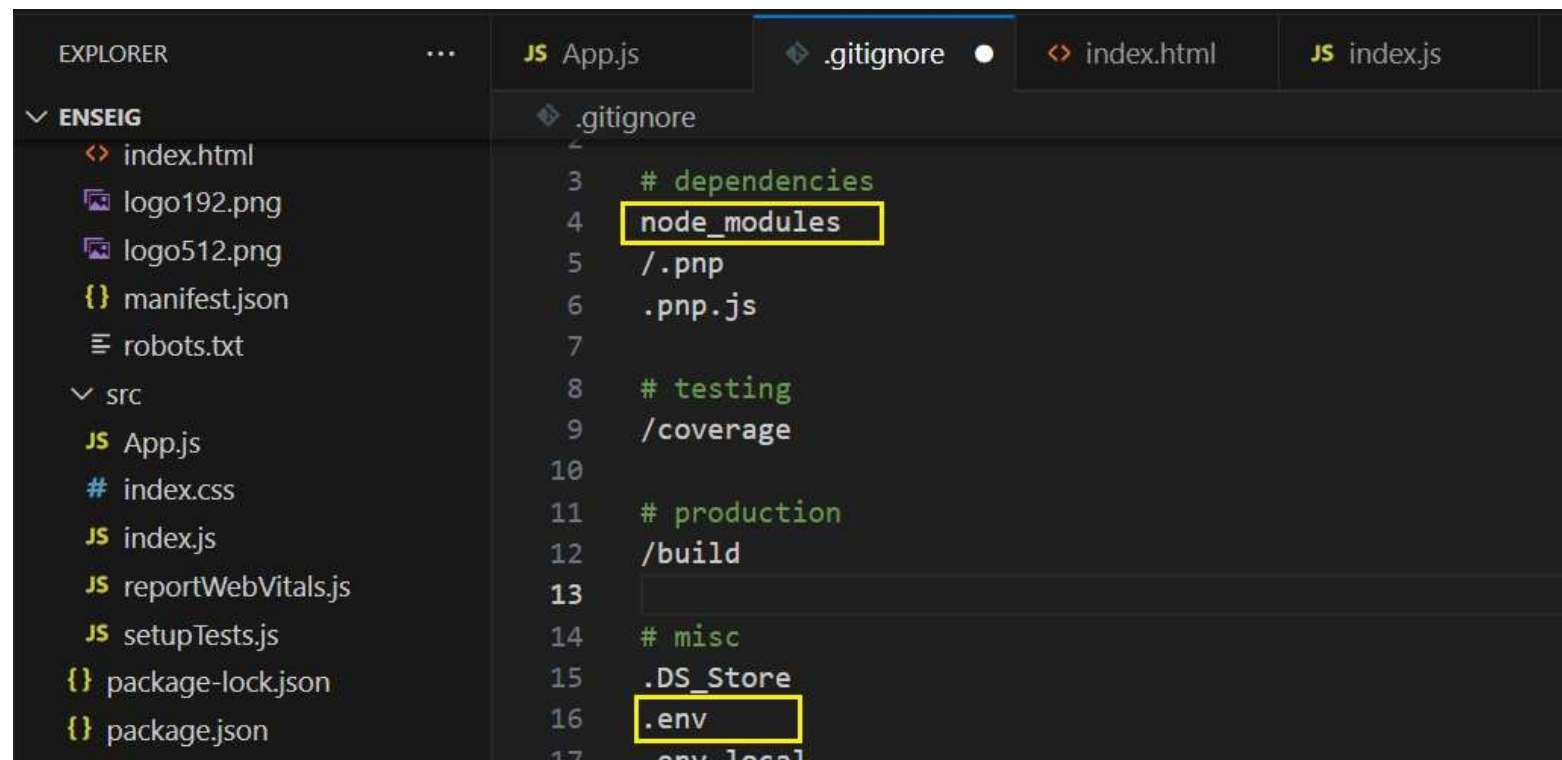
Adapter l'application pour Github

- Arrêter la compilation au niveau terminal de VS avec ctrl + c
- Configuration globale au début pour que ca marche avec git, taper dans le terminal de VS (remplacer nom et mail@mail.com) :
git config --global user.name "nom"
git config --global user.email mail@mail.com

Adapter l'application pour Github

- Ouvrir le GitBash et se positionner avec `cd` et `ls` dans le dossier frontend du projet
- Supprimer le dossier `.git` dans frontend (`rm -rf .git`) parce qu'on aura besoin dans le dossier parent (dossier racine du projet)
- Déplacer le fichier `.gitignore` depuis le frontend vers le dossier parent et le modifier comme suit et enregistrer

.gitignore



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar shows a project named 'ENSEIG' with files like 'index.html', 'logo192.png', 'logo512.png', 'manifest.json', 'robots.txt', and a 'src' folder containing 'App.js', 'index.css', 'index.js', 'reportWebVitals.js', 'setupTests.js', 'package-lock.json', and 'package.json'. The main editor area has tabs for 'App.js', '.gitignore', 'index.html', and 'index.js'. The '.gitignore' tab is active, showing the following content:

```
1 #
2
3 # dependencies
4 node_modules
5 /.pnp
6 .pnp.js
7
8 # testing
9 /coverage
10
11 # production
12 /build
13
14 # misc
15 .DS_Store
16 .env
17 .env.local
```

In this file, the entries 'node_modules' on line 4 and '.env' on line 16 are highlighted with yellow boxes.

Adapter l'application pour Github

- Après initialiser un repository depuis la racine du projet, en lançant la commande suivante dans le terminal de VS code : `git init`
- Dans le Git Bash lancer `ls -a` pour vérifier que `.git` est créé dans la racine du projet
- Après, dans le navigateur, ouvrir notre compte Github, create a new repository et le nommer, le marquer privé, créer, et 3 lignes sont générées, exemple :
`git remote add origin git@github.com:tarek77/exemple.git`
`git branch -M main`
`git push -u origin main`

Adapter l'application pour Github

- Puis dans le terminal de VS code, taper : `git add .`
- Après taper : `git commit -m 'initial commit'`
- Puis copier et exécuter successivement les 3 lignes déjà générées par le repository dans votre compte Github
- Après on peut utiliser directement Source Control depuis VS pour ajouter de nouvelles modifications directement sur le repository dans votre compte Github (après modification et enregistrement du projet) ; en donnant le nom de la modification et cliquer sur commit

Créer le Header et le Footer

- Arrêter le terminal s'il est en marche avec ctrl + c
- D'abord dans le terminal, au niveau du dossier frontend ajouter les librairies suivantes : `npm i react-bootstrap bootstrap react-icons`
- Après modifier le fichier `index.js` et exécuter `npm start` dans le terminal (niveau frontend) et il y aura un changement du type de texte puisque qu'on a installé bootstrap

index.js

```
JS App.js    JS index.js M ●
frontend > src > JS index.js > ...
1  import React from 'react';
2  import ReactDOM from 'react-dom/client';
3  import 'bootstrap/dist/css/bootstrap.min.css';
4  import './index.css';
5  import App from './App';
6  import reportWebVitals from './reportWebVitals';
7
8  const root = ReactDOM.createRoot(document.getElementById('root'));
9  root.render(
10   <React.StrictMode>
11   |   <App />
12   </React.StrictMode>
13 );
14
15 reportWebVitals();
```

Créer le Header et le Footer

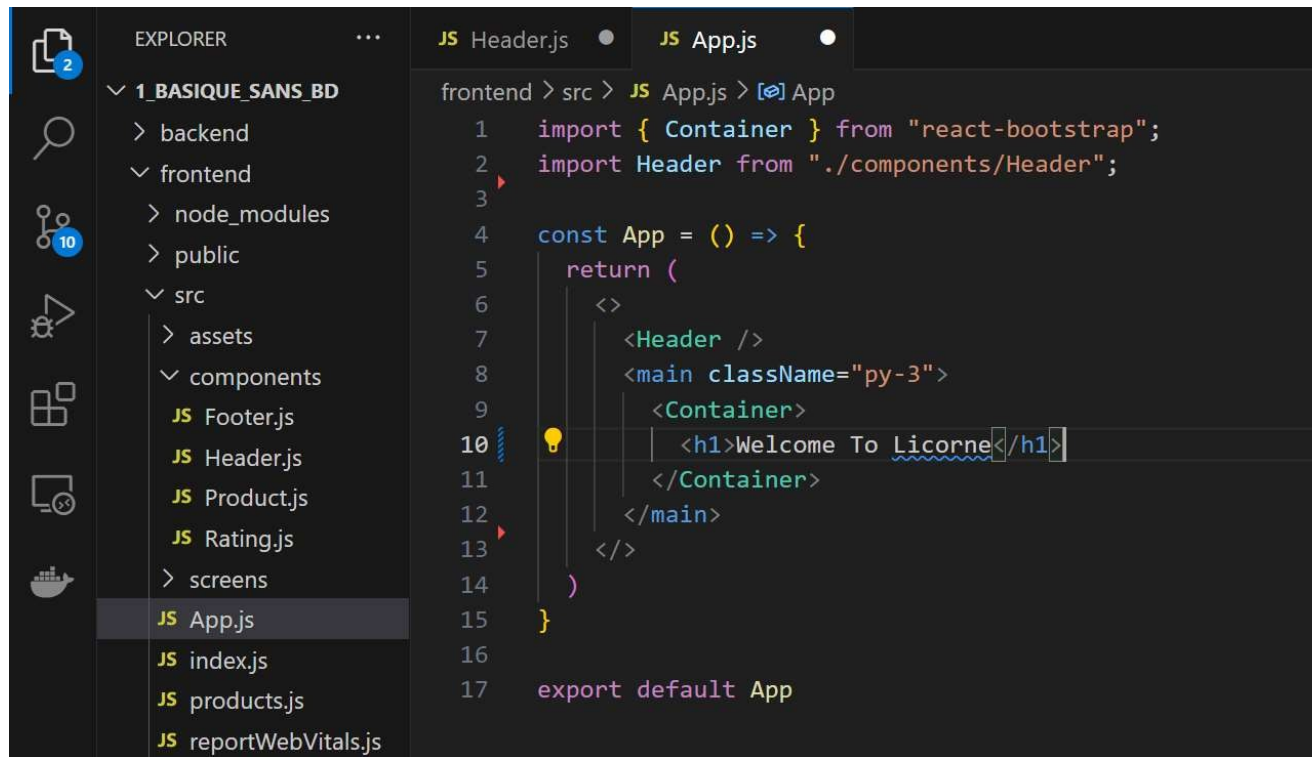
- Dans le dossier src, créer un dossier components, puis dans components créer un fichier Header.js, puis modifier le fichier App.js, puis créer dans components le fichier Footer.js, puis modifier encore App.js pour que le Footer.js marche
- Et exécuter, ca va nous afficher le résultat avec l'entete et le pied de page

Header.js

```
> backend
✓ frontend
  > node_modules
  > public
  ✓ src
    > assets
    ✓ components
      JS Footer.js
      JS Header.js
      JS Product.js
      JS Rating.js
    > screens
      JS App.js
      JS index.js
      JS products.js
      JS reportWebVitals.js
      JS setupTests.js
  {} package-lock.json
  {} package.json
> OUTLINE

1  import { Navbar, Nav, Container } from 'react-bootstrap';
2  import { FaShoppingCart, FaUser } from 'react-icons/fa';
3
4  const Header = () => {
5    return (
6      <header>
7        <Navbar bg="dark" variant="dark" expand="md" collapseOnSelect>
8          <Container>
9            <Navbar.Brand href="/">Licorne</Navbar.Brand>
10           <Navbar.Toggle aria-controls="basic-navbar-nav" />
11           <Navbar.Collapse id="basic-navbar-nav">
12             <Nav className="ms-auto">
13               <Nav.Link href="/cart"><FaShoppingCart/> Cart </Nav.Link>
14               <Nav.Link href="/login"><FaUser/> Sign in </Nav.Link>
15             </Nav>
16           </Navbar.Collapse>
17         </Container>
18       </Navbar>
19     </header>
20   )
21 }
22
23 export default Header
```

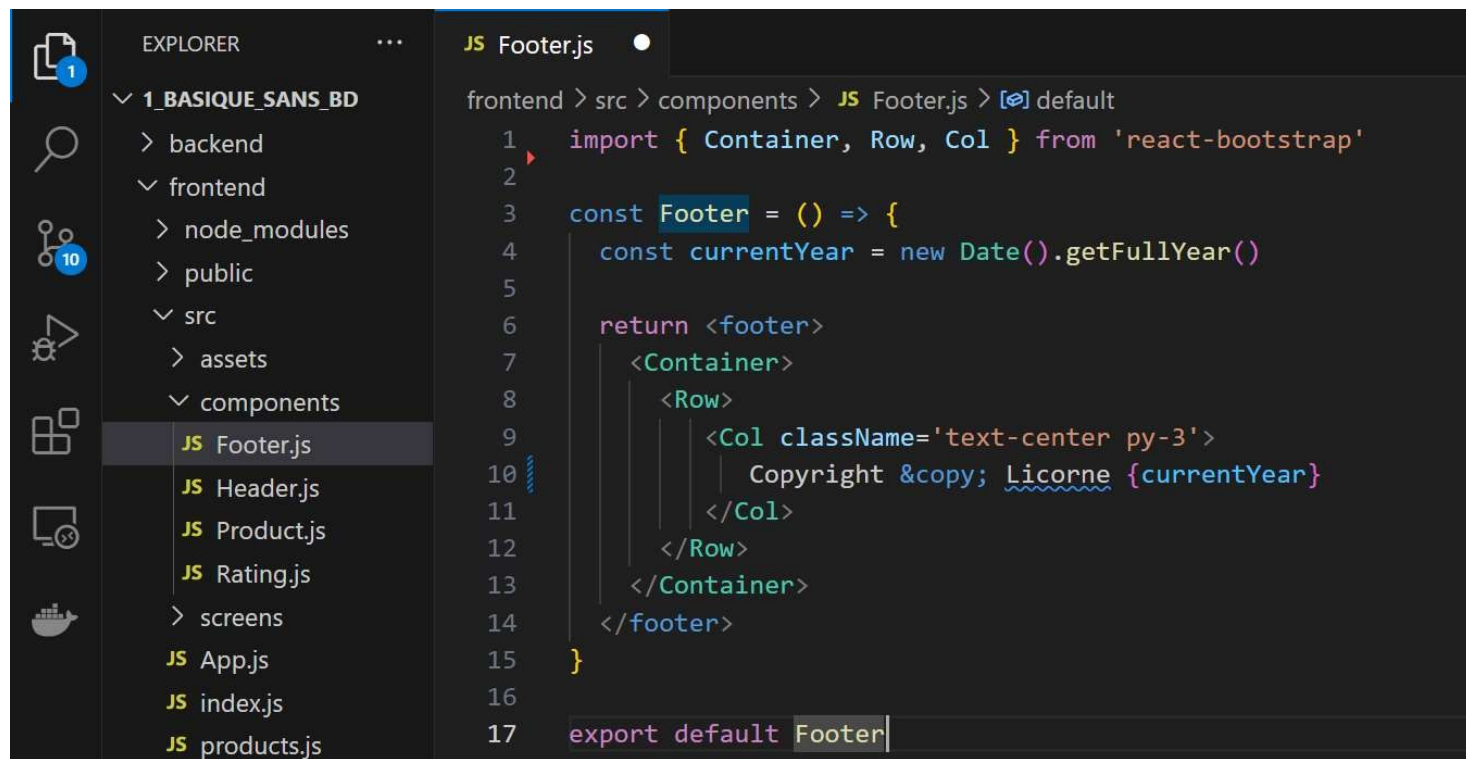
app.js



The screenshot shows the Visual Studio Code editor interface. On the left is the Explorer sidebar with a file tree for a project named '1_BASIQUE_SANS_BD'. The tree includes folders for 'backend', 'frontend', 'node_modules', 'public', and 'src'. The 'src' folder is expanded, showing subfolders 'assets', 'components', and 'screens'. The 'components' folder is further expanded, listing 'Footer.js', 'Header.js', 'Product.js', 'Rating.js', and 'App.js'. 'App.js' is selected and highlighted. The main editor area displays the code for 'App.js'. It starts with imports for 'Container' from 'react-bootstrap' and 'Header' from './components/Header'. A functional component 'App' is defined, which returns a JSX element. The JSX includes a 'Header' component, a 'main' element with 'className="py-3"', and a 'Container' element containing an 'h1' with the text 'Welcome To Licorne'. The 'h1' text has a blue wavy underline, indicating a link. The file is exported as the default export.

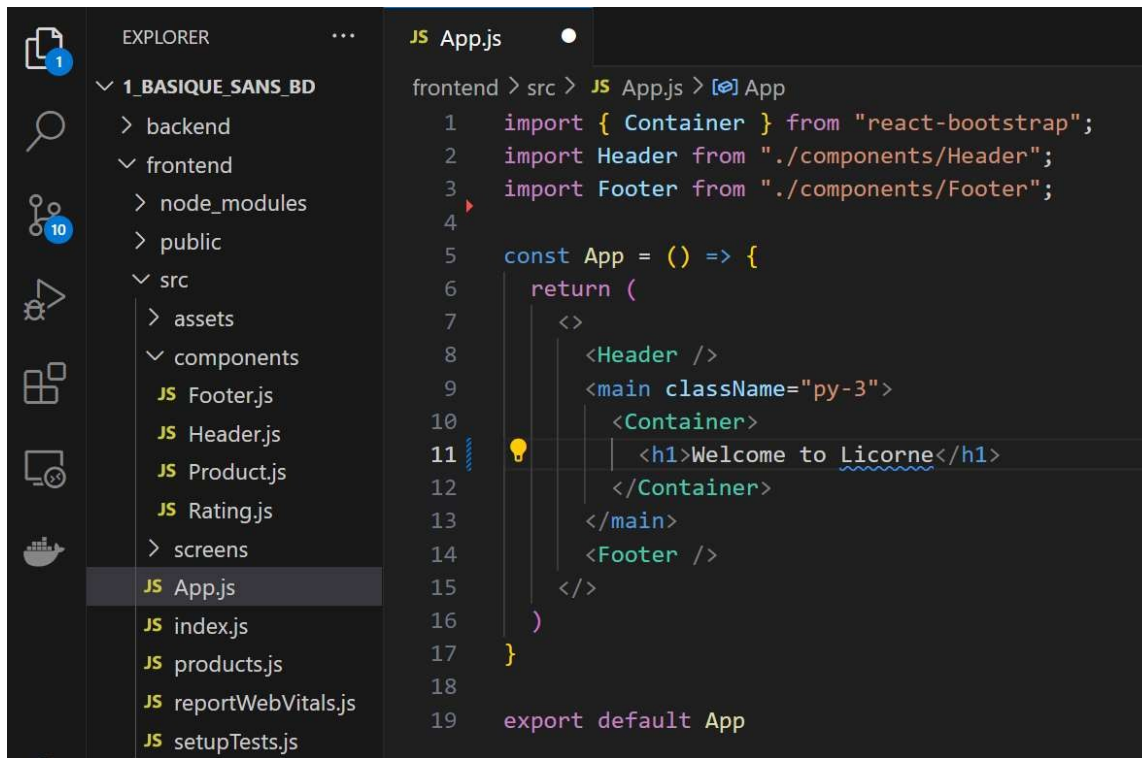
```
1 import { Container } from "react-bootstrap";
2 import Header from "../components/Header";
3
4 const App = () => {
5   return (
6     <>
7       <Header />
8       <main className="py-3">
9         <Container>
10          <h1>Welcome To Licorne</h1>
11        </Container>
12      </main>
13    </>
14  )
15 }
16
17 export default App
```

Footer.js



```
JS Footer.js
frontend > src > components > JS Footer.js > [default]
1 import { Container, Row, Col } from 'react-bootstrap'
2
3 const Footer = () => {
4   const currentYear = new Date().getFullYear()
5
6   return <footer>
7     <Container>
8       <Row>
9         <Col className='text-center py-3'>
10           Copyright &copy; Licorne {currentYear}
11         </Col>
12       </Row>
13     </Container>
14   </footer>
15 }
16
17 export default Footer
```

app.js



The screenshot shows a code editor with a dark theme. On the left is the 'EXPLORER' sidebar showing a file tree for a project named '1_BASIQUE_SANS_BD'. The tree includes folders for 'backend', 'frontend', 'node_modules', 'public', and 'src'. Inside 'src', there are folders for 'assets', 'components', and 'screens'. The 'components' folder contains files: 'Footer.js', 'Header.js', 'Product.js', 'Rating.js', and 'App.js' (which is selected). The 'App.js' file is open in the main editor area. The code in 'App.js' imports 'Container' from 'react-bootstrap' and 'Header' and 'Footer' from local components. It defines a functional component 'App' that returns a JSX element with a header, a main container with a class 'py-3' containing a 'Container' with a 'Welcome to Licorne' message, and a footer. The component is exported as the default export.

```
JS App.js
frontend > src > JS App.js > [App]
1  import { Container } from "react-bootstrap";
2  import Header from "../components/Header";
3  import Footer from "../components/Footer";
4
5  const App = () => {
6    return (
7      <>
8        <Header />
9        <main className="py-3">
10         <Container>
11           <h1>Welcome to Licorne</h1>
12         </Container>
13       </main>
14       <Footer />
15     </>
16   )
17 }
18
19 export default App
```

Créer le Header et le Footer

- Dans le dossier src, créer un dossier assets, dans lequel on va copier un fichier image logo.png, et on va créer un dossier styles dans le quel on va mettre deux fichiers CSS : bootstrap.custom.css et index.css, pour pouvoir styliser d'une manière personnalisée notre projet.
- Supprimer le fichier index.css dans le dossier src parce qu'on ne va plus l'utiliser
- Dans le Header.js, ajouter cette importation en haut :
`import logo from '../assets/logo.png'`
- Dans le Header.js, modifier la ligne `</Navbar.Brand>` comme suit :
`</Navbar.Brand href='/'> <img src={logo} alt='Licorne' /> Licorne`
`</NavBar.Brand>`
- Après modifier index.js et exécuter

index.js

```
JS App.js JS index.js M X JS Header.js M JS Footer.js
frontend > src > JS index.js > ...
1 import React from 'react';
2 import ReactDOM from 'react-dom/client';
3
4 //import 'bootstrap/dist/css/bootstrap.min.css';
5 import './assets/styles/bootstrap.custom.css'
6 import './assets/styles/index.css';
7
8 import App from './App';
9 import reportWebVitals from './reportWebVitals';
10
11 const root = ReactDOM.createRoot(document.getElementById('root'));
12 root.render(
13   <React.StrictMode>
14     <App />
15   </React.StrictMode>
16 );
17
18 reportWebVitals();
19
```

Liste de produits - statique

- Dans le dossier public, copier le dossier images (contient les images des produits), puis dans le dossier src, copier le fichier products.js (contient les informations sur les produits)
- Dans le dossier src créer un dossier screens, et dans le dossier screens, créer un fichier HomeScreen.js
- Puis modifier encore App.js en important en haut :
`import HomeScreen from './screens/HomeScreen';`
- Dans App.js et après `<Container>` ajouter :
`<HomeScreen />`
- Et exécuter, ca va nous afficher le titres des produits

HomeScreen.js

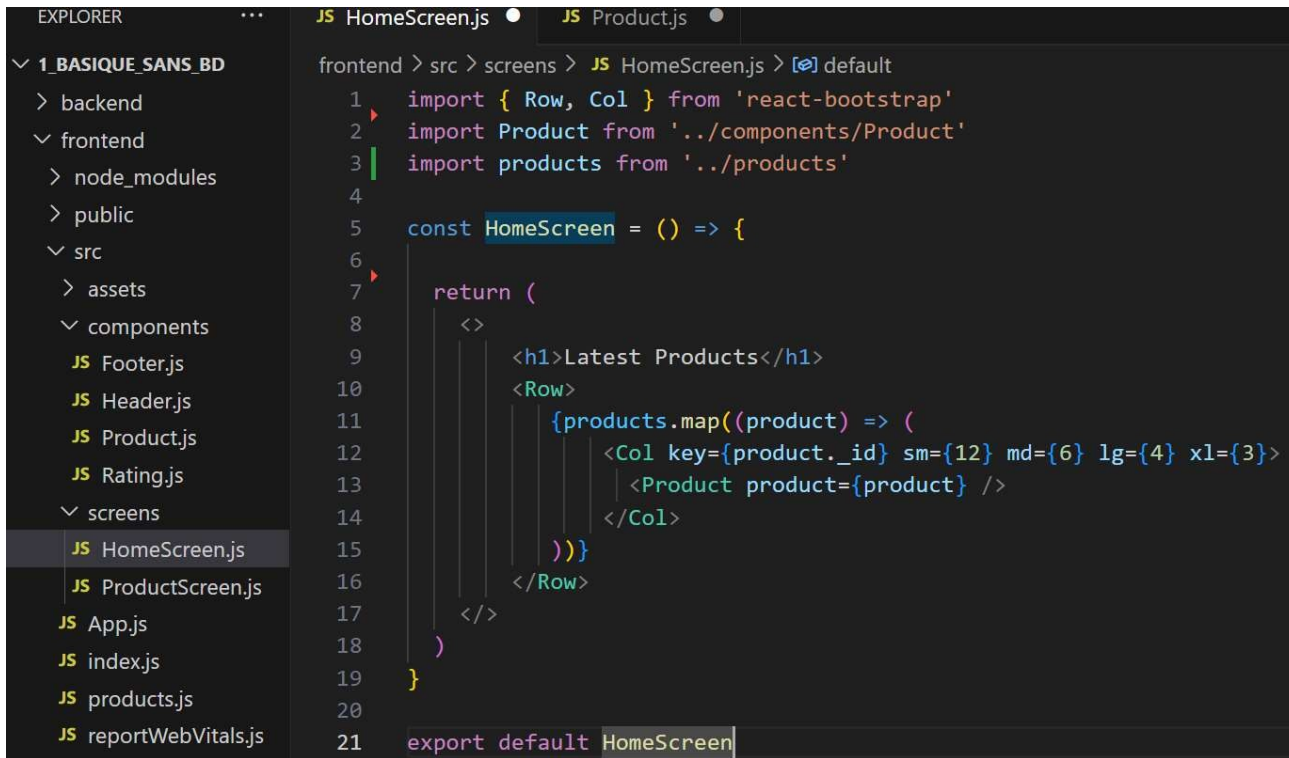
```
EXPLORER  ...
1_BASIQUE_SANS_BD
  > backend
  > frontend
    > node_modules
    > public
    > src
      > assets
        > styles
        logo.png
      > components
      > screens
        JS HomeScreen.js
        JS ProductScreen.js
        JS App.js
        JS index.js
        JS products.js
        JS reportWebVitals.js
        JS setupTests.js
        JS ...

JS HomeScreen.js
frontend > src > screens > JS HomeScreen.js > HomeScreen > products.map() callback
1  import { Row, Col } from 'react-bootstrap'
2  import products from '../products'
3
4  const HomeScreen = () => {
5
6    return (
7      <>
8        <h1>Latest Products</h1>
9        <Row>
10         {products.map((product) => (
11           <Col key={product._id} sm={12} md={6} lg={4} xl={3}>
12             <h3>{product.name}</h3>
13           </Col>
14         ))}
15       </Row>
16     </>
17   )
18 }
19
20 export default HomeScreen
```

Liste de produits - statique

- Maintenant, on va afficher toutes les informations sur les produits, on va aller dans le dossier components et créer le fichier Product.js
- Dans le fichier Product.js, on va créer un props (comme une variable d'entrée envoyée depuis un autre fichier js, ici depuis HomeScreen.js)
- Dans le fichier HomeScreen.js, ajouter en haut le fichier Product.js (voir l'imprime écran en bas)
- Dans le fichier HomeScreen, faire passer le props product pour le fichier Product.js (modifier le h3, voir l'imprime écran en bas)
- Dans le fichier HomeScreen, ajouter la clé id (voir l'imprime écran en bas)
- Et exécuter, ca va nous afficher les informations sur les produits

HomeScreen.js



```
EXPLORER
1_BASIQUE_SANS_BD
  backend
  frontend
    node_modules
    public
    src
      assets
      components
        Footer.js
        Header.js
        Product.js
        Rating.js
      screens
        JS HomeScreen.js
        JS ProductScreen.js
      App.js
      index.js
      products.js
      reportWebVitals.js

JS HomeScreen.js
JS Product.js

frontend > src > screens > JS HomeScreen.js > [0] default
1  import { Row, Col } from 'react-bootstrap'
2  import Product from '../components/Product'
3  import products from '../products'
4
5  const HomeScreen = () => {
6
7    return (
8      <>
9        <h1>Latest Products</h1>
10       <Row>
11         {products.map((product) => (
12           <Col key={product._id} sm={12} md={6} lg={4} xl={3}>
13             <Product product={product} />
14           </Col>
15         ))}
16       </Row>
17     </>
18   )
19 }
20
21 export default HomeScreen
```

Product.js

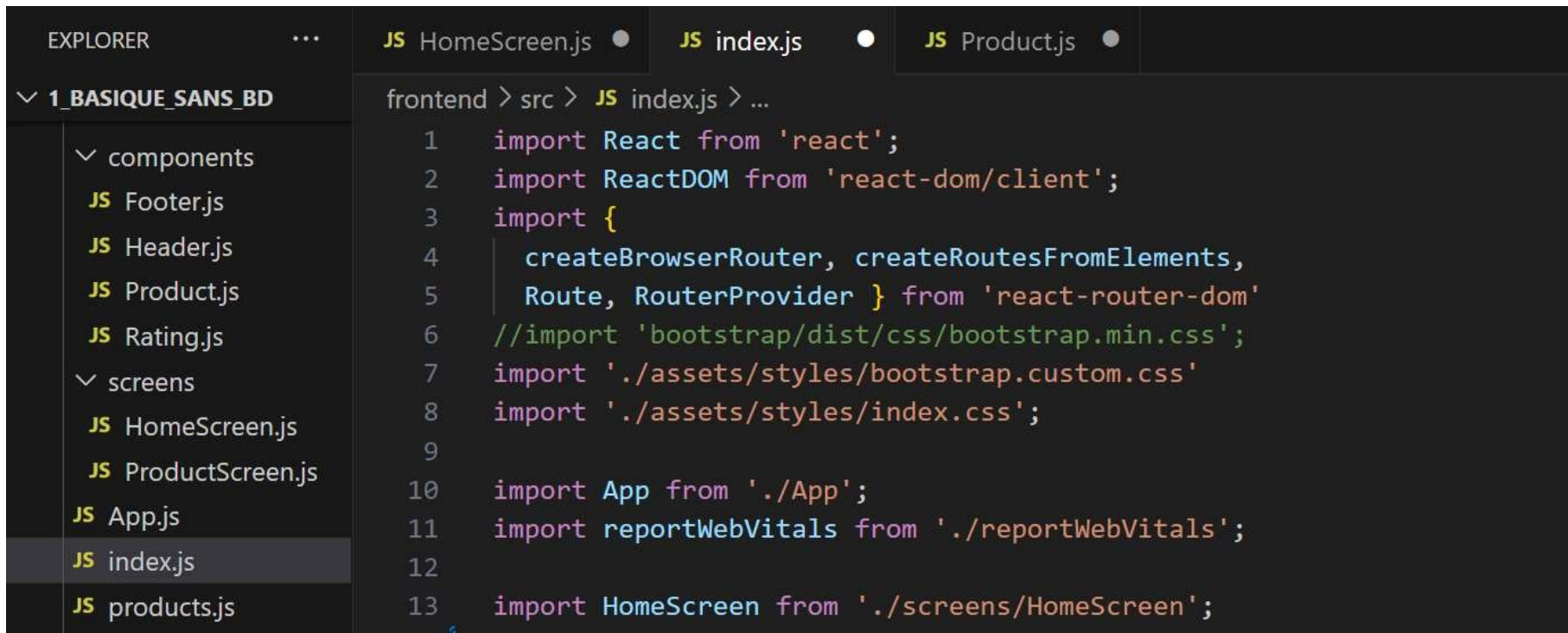
```
> backend
  > frontend
    > node_modules
    > public
    > src
      > assets
      > components
        JS Footer.js
        JS Header.js
        JS Product.js
        JS Rating.js
      > screens
        JS HomeScreen.js
        JS ProductScreen.js
      JS App.js
      JS index.js
      JS products.js
      JS reportWebVitals.js
      JS setupTests.js
    > OUTLINE
    > TIMELINE

1  import { Card } from 'react-bootstrap'
2
3  const Product = ({ product }) => {
4    return (
5      <Card className='my-3 p-3 rounded'>
6        <a href={` /product/${product._id}`} >
7          <Card.Img src={product.image} variant='top' />
8        </a>
9
10       <Card.Body>
11         <a href={` /product/${product._id}`} >
12           <Card.Title className='product-title' as='div' >
13             <strong>{product.name}</strong>
14           </Card.Title>
15         </a>
16
17         <Card.Text as='h3'>${product.price}
18       </Card.Text>
19     </Card.Body>
20   </Card>
21 )
22 }
23
24 export default Product
```

Implémenter le React Router pour les liens

- Maintenant, arrêter le serveur, et dans le terminal niveau frontend :
npm i react-router-dom
- Avec le React Router la page ne se charge plus comme avec href
- On trouve qu'il a ajouté la bibliothèque dans le fichier package.json
- Exécuter le serveur avec npm start
- Après modifier index.js en important le React Router
- Et modifier app.js pour l'adapter au React Router
- Et exécuter, ca va nous afficher le même résultat, mais les pages ne se chargent pas comme avec href

index.js



The screenshot shows a code editor with a dark theme. On the left is the 'EXPLORER' sidebar showing a project structure under '1_BASIQUE_SANS_BD'. It includes folders 'components' and 'screens', and several JavaScript files. The 'index.js' file is selected. The main editor area shows the code for 'index.js' with line numbers 1 through 13. The code imports React, ReactDOM, and various components from 'react-router-dom', as well as CSS files and other modules like 'App', 'reportWebVitals', and 'HomeScreen'.

```
EXPLORER  ...  JS HomeScreen.js  JS index.js  JS Product.js

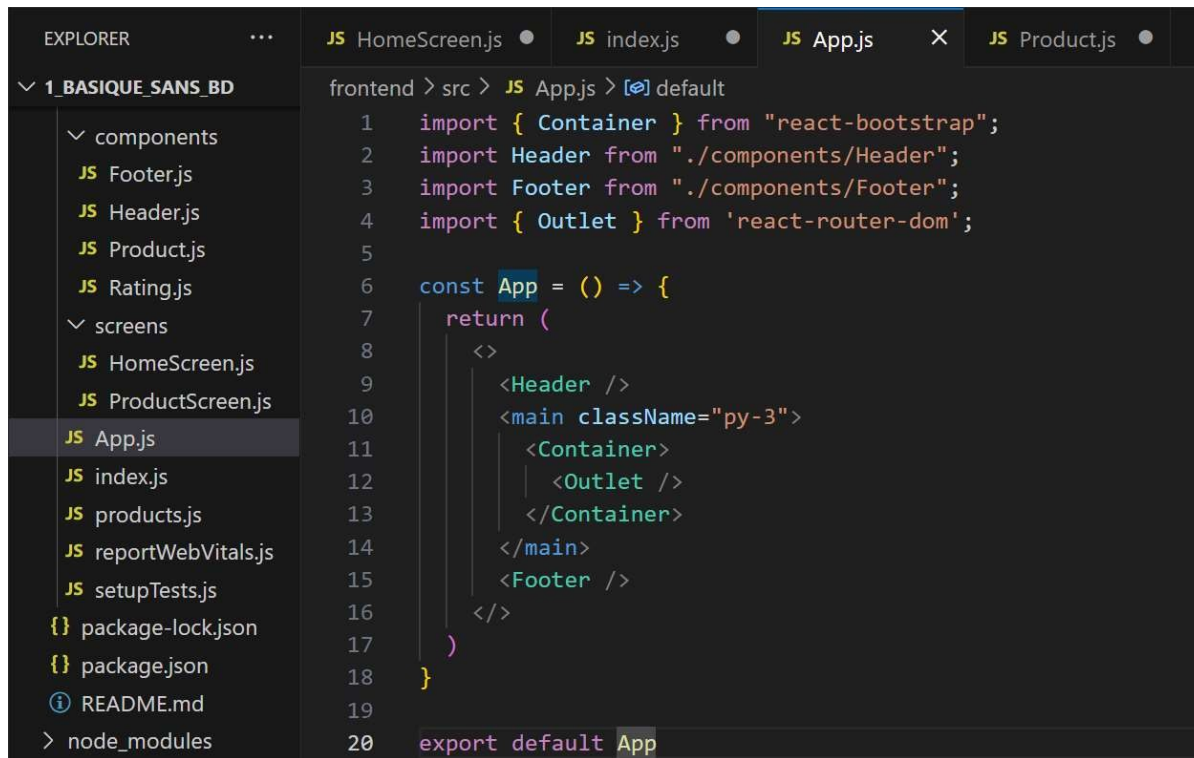
▼ 1_BASIQUE_SANS_BD
  ▼ components
    JS Footer.js
    JS Header.js
    JS Product.js
    JS Rating.js
  ▼ screens
    JS HomeScreen.js
    JS ProductScreen.js
  JS App.js
  JS index.js
  JS products.js

frontend > src > JS index.js > ...
1  import React from 'react';
2  import ReactDOM from 'react-dom/client';
3  import {
4    createBrowserRouter, createRoutesFromElements,
5    Route, RouterProvider } from 'react-router-dom'
6  //import 'bootstrap/dist/css/bootstrap.min.css';
7  import './assets/styles/bootstrap.custom.css'
8  import './assets/styles/index.css';
9
10 import App from './App';
11 import reportWebVitals from './reportWebVitals';
12
13 import HomeScreen from './screens/HomeScreen';
```

index.js

```
JS Footer.js      16  const router = createBrowserRouter(  
JS Header.js      17    createRoutesFromElements(  
JS Product.js     18      <Route path="/" element={<App />}>  
JS Rating.js      19        <Route index={true} path="/" element={<HomeScreen />} />  
  screens         20        <Route path="/product/:id" element={<ProductScreen />} />  
JS HomeScreen.js  21      </Route>  
JS ProductScreen.js 22    )  
JS App.js         23  )  
JS index.js       24  
JS products.js    25  const root = ReactDOM.createRoot(document.getElementById('root'));  
JS reportWebVitals.js 26  root.render(  
JS setupTests.js  27    <React.StrictMode>  
  package-lock.json 28      <RouterProvider router={router} />  
  package.json      29    </React.StrictMode>  
  README.md         30  );  
                   31  
                   32  reportWebVitals();
```

app.js



```
EXPLOLERER  ...  JS HomeScreen.js  JS index.js  JS App.js  JS Product.js

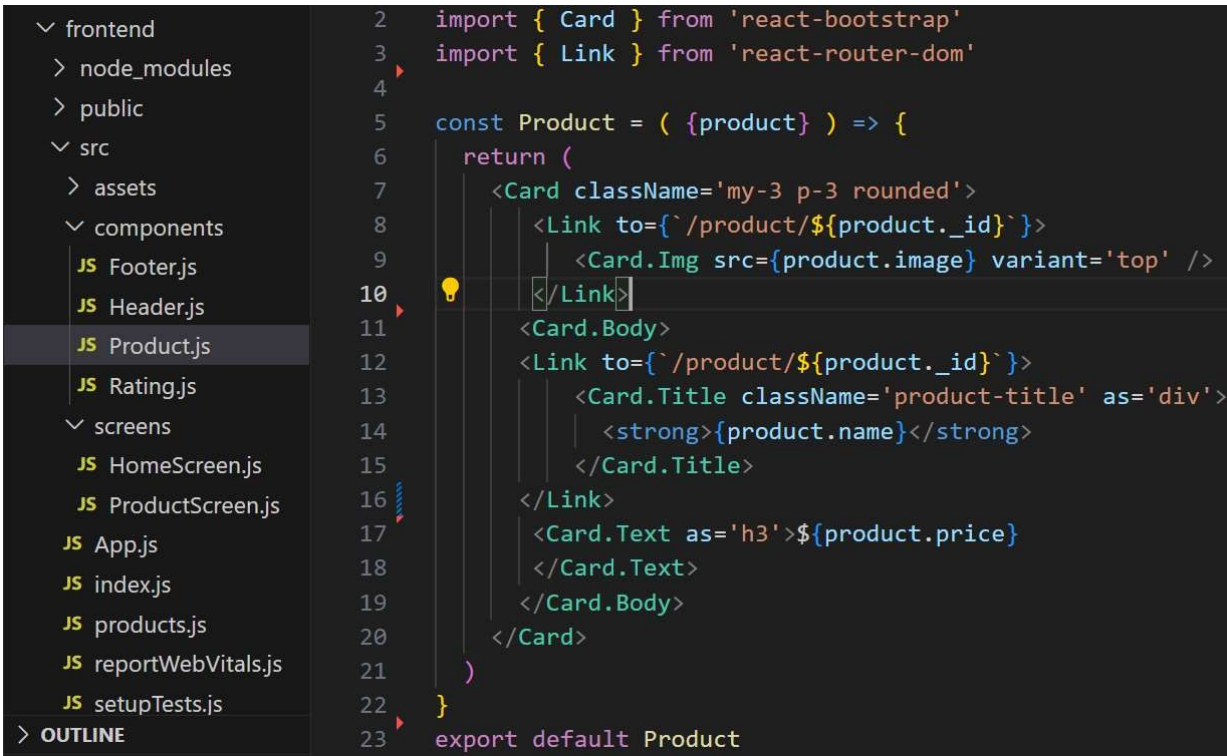
1_BASIQUE_SANS_BD
├── components
│   ├── JS Footer.js
│   ├── JS Header.js
│   ├── JS Product.js
│   └── JS Rating.js
├── screens
│   ├── JS HomeScreen.js
│   └── JS ProductScreen.js
├── JS App.js
├── JS index.js
├── JS products.js
├── JS reportWebVitals.js
├── JS setupTests.js
├── {} package-lock.json
├── {} package.json
├── ⓘ README.md
└── > node_modules

frontend > src > JS App.js > [🔗] default
1  import { Container } from "react-bootstrap";
2  import Header from "../components/Header";
3  import Footer from "../components/Footer";
4  import { Outlet } from "react-router-dom";
5
6  const App = () => {
7      return (
8          <>
9              <Header />
10             <main className="py-3">
11                 <Container>
12                     <Outlet />
13                 </Container>
14             </main>
15             <Footer />
16         </>
17     )
18 }
19
20 export default App
```

Implémenter le React Router pour les liens

- Aller modifier Product.js et remplacer les a href par des Link
- Et exécuter, ca va nous afficher le même résultat, mais les pages ne se chargent pas comme avec href

Product.js

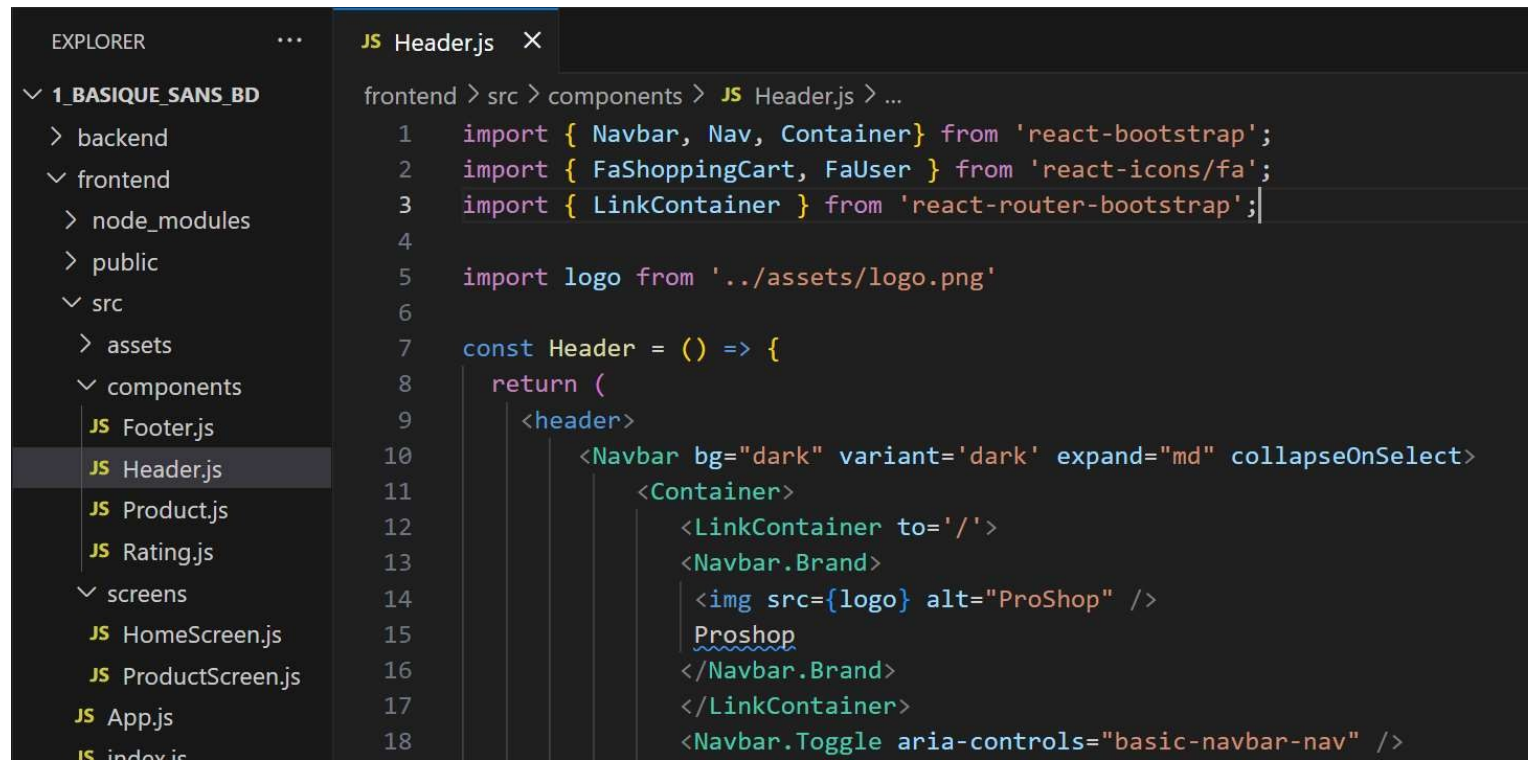


```
2 import { Card } from 'react-bootstrap'
3 import { Link } from 'react-router-dom'
4
5 const Product = ( {product} ) => {
6   return (
7     <Card className='my-3 p-3 rounded'>
8       <Link to={` /product/${product._id}`}>
9         <Card.Img src={product.image} variant='top' />
10      </Link>
11     <Card.Body>
12       <Link to={` /product/${product._id}`}>
13         <Card.Title className='product-title' as='div'>
14           <strong>{product.name}</strong>
15         </Card.Title>
16       </Link>
17       <Card.Text as='h3'>${product.price}
18     </Card.Text>
19   </Card.Body>
20 </Card>
21 )
22 }
23 export default Product
```

Implémenter le React Router pour les liens

- Après arrêter le serveur et dans le terminal niveau frontend :
npm i react-router-bootstrap
- Après lancer le serveur : npm start
- Et modifier le Header.js :
importer le LinkContainer et modifier les href avec des LinkContainer
- Et exécuter, ca va nous afficher le même résultat, mais les pages ne se chargent pas comme avec href

Header.js



```
EXPLORER  ...
1_BASIQUE_SANS_BD
├── backend
├── frontend
│   ├── node_modules
│   ├── public
│   ├── src
│   │   ├── assets
│   │   └── components
│   │       ├── JS Footer.js
│   │       ├── JS Header.js
│   │       ├── JS Product.js
│   │       ├── JS Rating.js
│   │       └── screens
│   │           ├── JS HomeScreen.js
│   │           ├── JS ProductScreen.js
│   │           ├── JS App.js
│   │           └── JS index.js
└── ...

JS Header.js X
frontend > src > components > JS Header.js > ...
1  import { Navbar, Nav, Container } from 'react-bootstrap';
2  import { FaShoppingCart, FaUser } from 'react-icons/fa';
3  import { LinkContainer } from 'react-router-bootstrap';
4
5  import logo from '../assets/logo.png'
6
7  const Header = () => {
8      return (
9          <header>
10             <Navbar bg="dark" variant='dark' expand="md" collapseOnSelect>
11                 <Container>
12                     <LinkContainer to="/">
13                         <Navbar.Brand>
14                             <img src={logo} alt="ProShop" />
15                             Proshop
16                         </Navbar.Brand>
17                     </LinkContainer>
18                     <Navbar.Toggle aria-controls="basic-navbar-nav" />
```

Header.js

```
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
```

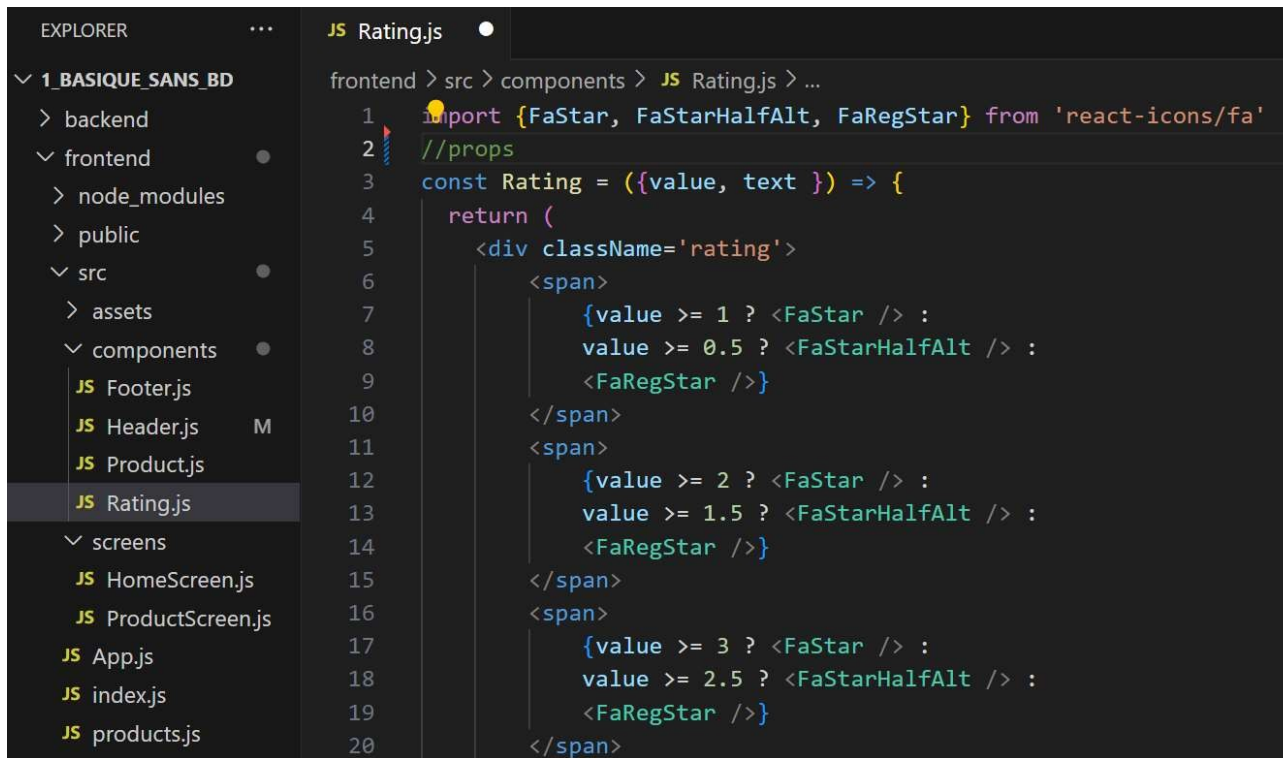
```
<Navbar.Collapse id="basic-navbar-nav">
  <Nav className="ms-auto">
    <LinkContainer to="/cart">
      <Nav.Link><FaShoppingCart/> Cart </Nav.Link>
    </LinkContainer>
    <LinkContainer to="/login">
      <Nav.Link><FaUser/> Sign In </Nav.Link>
    </LinkContainer>
  </Nav>
</Navbar.Collapse>
</Container>
</Navbar>
</header>
)
}

export default Header
```

Afficher les étoiles pour un produit

- Créer le fichier Rating.js dans le dossier components

Rating.js

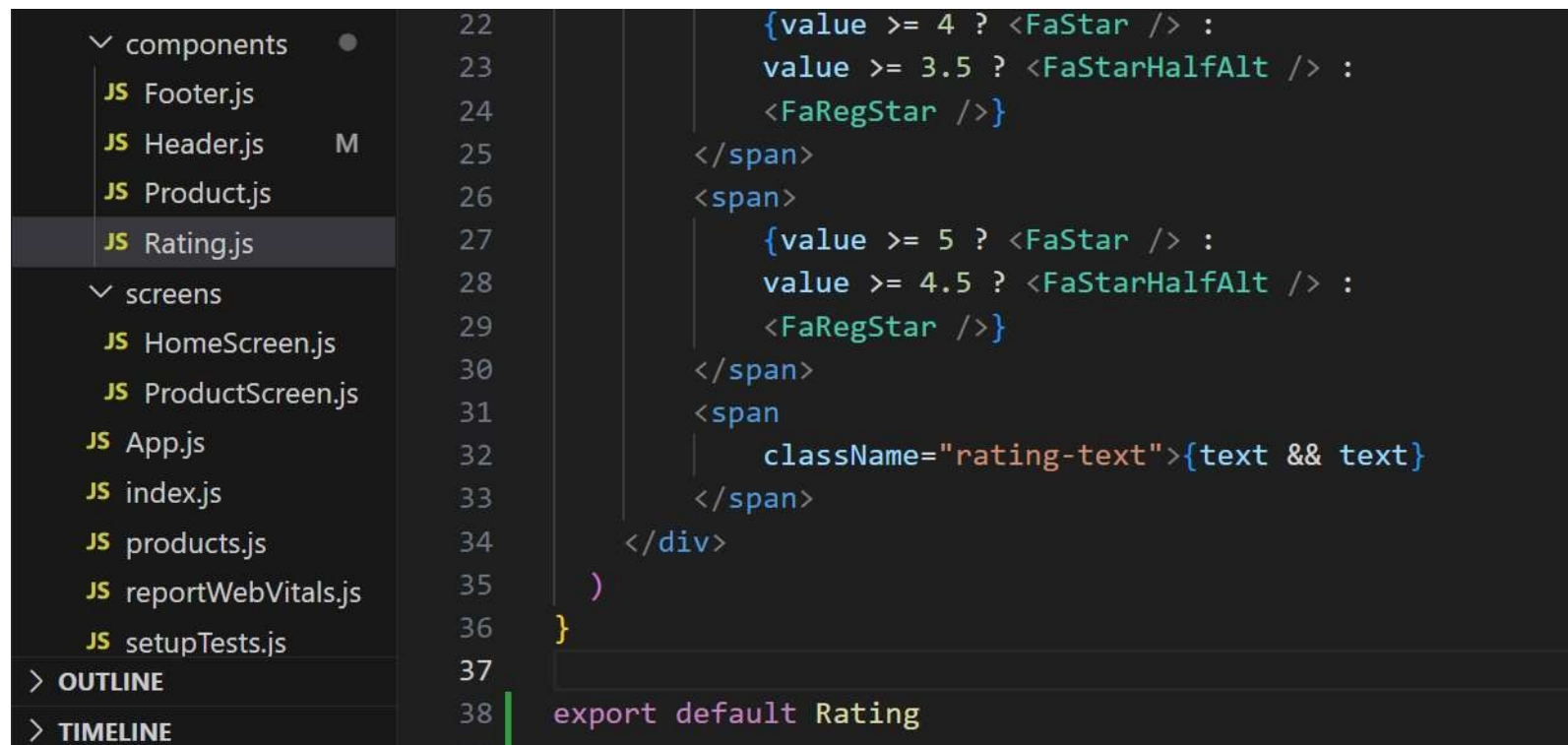


The image shows a screenshot of a code editor (VS Code) with a dark theme. On the left, the 'EXPLORER' sidebar shows a project structure under the name '1_BASIQUE_SANS_BD'. The 'components' folder is expanded, and 'Rating.js' is selected. The main editor area shows the code for 'Rating.js'. The code imports 'FaStar', 'FaStarHalfAlt', and 'FaRegStar' from 'react-icons/fa'. It defines a 'Rating' component as a function that takes 'value' and 'text' props. The component returns a JSX element with a 'div' of class 'rating' containing three 'span' elements. Each 'span' contains a conditional rendering of star icons based on the 'value' prop: 1 or more stars use 'FaStar', 0.5 or more but less than 1 use 'FaStarHalfAlt', and less than 0.5 use 'FaRegStar'. The code is numbered from 1 to 20.

```
1 import {FaStar, FaStarHalfAlt, FaRegStar} from 'react-icons/fa'
2 //props
3 const Rating = ({value, text }) => {
4   return (
5     <div className='rating'>
6       <span>
7         {value >= 1 ? <FaStar /> :
8         value >= 0.5 ? <FaStarHalfAlt /> :
9         <FaRegStar />}
10      </span>
11      <span>
12        {value >= 2 ? <FaStar /> :
13        value >= 1.5 ? <FaStarHalfAlt /> :
14        <FaRegStar />}
15      </span>
16      <span>
17        {value >= 3 ? <FaStar /> :
18        value >= 2.5 ? <FaStarHalfAlt /> :
19        <FaRegStar />}
20    </span>

```

Rating.js



The screenshot shows an IDE with a project structure on the left and a code editor on the right. The project structure includes a 'components' folder with 'Footer.js', 'Header.js', 'Product.js', and 'Rating.js' (selected), and a 'screens' folder with 'HomeScreen.js', 'ProductScreen.js', 'App.js', 'index.js', 'products.js', 'reportWebVitals.js', and 'setupTests.js'. The code editor displays the following JavaScript code for 'Rating.js':

```
22     {value >= 4 ? <FaStar /> :  
23     value >= 3.5 ? <FaStarHalfAlt /> :  
24     <FaRegStar />}  
25   </span>  
26   <span>  
27     {value >= 5 ? <FaStar /> :  
28     value >= 4.5 ? <FaStarHalfAlt /> :  
29     <FaRegStar />}  
30   </span>  
31   <span  
32     className="rating-text">{text && text}  
33   </span>  
34 </div>  
35 )  
36 }  
37  
38 export default Rating
```

Afficher les étoiles pour un produit

- Après dans Product.js :
`import Rating from './Rating';`
- Toujours dans Product.js, avant l'affichage du prix (price) :
`<Card.Text as='div'>
 <Rating value={product.rating} text={` ${product.numReviews} reviews` } />
</Card.Text>`
- Et exécuter, ca va afficher les étoiles d'évaluation pour chaque produit

Afficher les détails d'un seul produit

- Créer un nouveau fichier ProductScreen.js dans le dossier screens (ci-dessus), sachant que useParams permet de récupérer un objet du paramètre dynamique de l'URL en cours (ici il va récupérer l'id du produit)
- Dans index.js ajouter :

```
import ProductScreen from './screens/ProductScreen';
```
- Dans index.js, et après le Route de HomeScreen, ajouter :

```
<Route path="/product/:id" element={ <ProductScreen /> } />
```
- Et exécuter, on obtient l'affichage des informations pour un seul produit

ProductScreen.js

```
EXPLORER  ...
1_BASIQUE_SANS_BD
  > backend
  > frontend
  > node_modules
  > public
  > src
  > assets
  > components
    JS Footer.js
    JS Header.js M
    JS Product.js
    JS Rating.js
  > screens
    JS HomeScreen.js
    JS ProductScreen.js
  JS App.js

JS ProductScreen.js
frontend > src > screens > JS ProductScreen.js > ...
1
2 import { useParams, Link } from 'react-router-dom'
3 import { Row, Col, Image, ListGroup, Card, Button } from 'react-bootstrap'
4 import products from '../products'
5 import Rating from '../components/Rating';
6
7 const ProductScreen = () => {
8   const { id: productId } = useParams();
9   const product = products.find((p) => p._id === productId)
10   return(
11     <>
12       <Link className='btn btn-light my-3' to='/'>
13         Go Back
14       </Link>
15       <Row>
16         <Col md={5}>
17           <Image src={product.image} alt={product.name} fluid />
18         </Col>
```

ProductScreen.js

> backend	19			<Col md={4}>
✓ frontend	20			<ListGroup variant='flush'>
> node_modules	21			<ListGroup.Item>
> public	22			{product.name}
✓ src	23			</ListGroup.Item>
> assets	24			<ListGroup.Item>
✓ components	25			<Rating value={product.rating} text={` \${product.numReviews} reviews`} />
JS Footer.js	26			</ListGroup.Item>
JS Header.js	27			<ListGroup.Item>
JS Product.js	28			Price: \${product.price}
JS Rating.js	29			</ListGroup.Item>
✓ screens	30			<ListGroup.Item>
JS HomeScreen.js	31			Description: {product.description}
JS ProductScreen.js	32			</ListGroup.Item>
	33			</ListGroup>
	34			</Col>

ProductScreen.js

```

> backend 35
  frontend 36
    node_modules 37
    public 38
    src 39
      assets 40
      components 41
        Footer.js 42
        Header.js 43
        Product.js 44
        Rating.js 45
      screens 46
        HomeScreen.js 47
        ProductScreen.js 48
        App.js 49
        index.js 50
        products.js 51
  
```

```

<Col md={3}>
  <Card>
    <ListGroup variant='flush'>
      <ListGroup.Item>
        <Row>
          <Col>Price:</Col>
          <Col>
            <strong>${product.price}</strong>
          </Col>
        </Row>
      </ListGroup.Item>
      <ListGroup.Item>
        <Row>
          <Col>Status:</Col>
          <Col>
            <strong>{product.countInStock > 0 ? 'In Stock' :
              'Out of Stock'}</strong>
          </Col>
        </Row>
      </ListGroup.Item>
    </ListGroup>
  </Card>
</Col>
  
```

ProductScreen.js

```
55     <ListGroup.Item>
56       <Button
57         className='btn-block'
58         type='button'
59         disabled={product.countInStock === 0}
60       >
61         Add To cart
62       </Button>
63     </ListGroup.Item>
64   </ListGroup>
65 </Card>
66 </Col>
67 </Row>
68 </>
69 );
70 };
71
72 export default ProductScreen
```

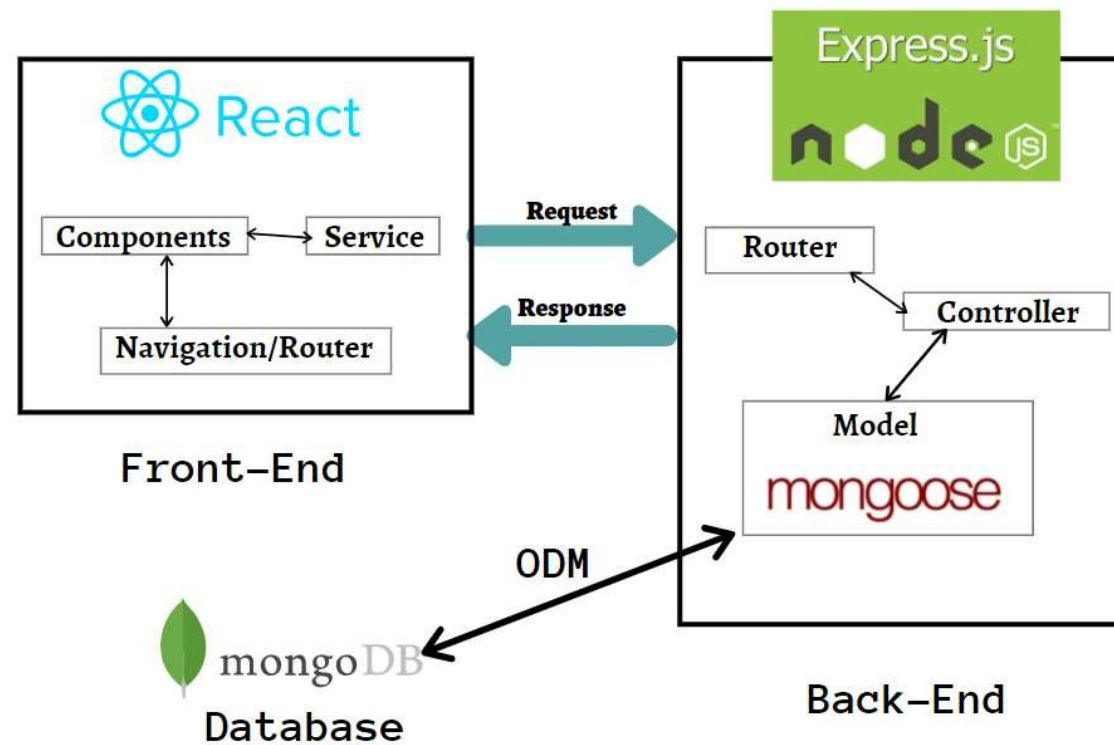
ProductScreen.js

```
55     <ListGroup.Item>
56       <Button
57         className='btn-block'
58         type='button'
59         disabled={product.countInStock === 0}
60       >
61         Add To cart
62       </Button>
63     </ListGroup.Item>
64   </ListGroup>
65 </Card>
66 </Col>
67 </Row>
68 </>
69 );
70 };
71
72 export default ProductScreen
```

Niveau 1-2 : Débutant confirmé

Backend
&
Frontend et Backend

Fonctionnement en Full Stack



Créer le Backend et initialiser les routes

- Arrêter le serveur, aller dans le terminal niveau racine et lancer :
`npm init`

indiquer la version, la description, entry point : `server.js`, le reste n'est pas important, indiquer le nom de l'auteur, Licence : MIT, et yes à la fin

- Un fichier `package.json` sera créé dans le root (racine)
- Ouvrir le fichier `package.json` et sous la description, ajouter :
`"type": "module",`

Créer le Backend et initialiser les routes

- Dans la racine, créer un dossier backend, puis dans backend créer le fichier server.js
- Dans le fichier server.js, écrire :
`console.log('Hello World');`
- Et dans le fichier package.json (dans le root), changer la ligne de test, en start :
`"start": "node backend/server.js"`
- Dans la racine du projet, dans le terminal, exécuter :
`npm start` et il va afficher dans le terminal un message

Créer le Backend et initialiser les routes

- Maintenant arrêtons le serveur, et dans le terminal niveau root, lancer l'installation du serveur web express :

`npm i express`

- Modifier le fichier `server.js` (page en bas)
- Et exécuter dans le terminal niveau root : `npm start`

Ca va nous afficher que le serveur tourne sur le port 5000

- Si j'ouvre un navigateur et je met l'adresse : `localhost:5000`, on nous affiche un message que ca marche

server.js

JS server.js

backend > JS server.js > ...

```
1  import express from 'express';
2  const port = 5000;
3
4  const app = express();
5
6  app.get('/', (req, res) => {
7    res.send('API is running...')
8  });
9
10 app.listen(port, () => console.log(`Server running on port ${port}`))
11
```

Créer le Backend et initialiser les routes

- Maintenant on va copier le fichier products.js (contenant les informations sur les produits) depuis le frontend/src vers le backend/data
- Aller modifier server.js (page en bas)
- Et lancer depuis le terminal, niveau root : npm start
- Si on ouvre un navigateur et on met l'adresse : localhost:5000/api/products ca va nous afficher les informations textuelles des produits

server.js

```
JS server.js ●
backend > JS server.js > ...
1  import express from 'express';
2  import products from './data/products.js';
3
4  const port = 5000;
5
6  const app = express();
7
8  app.get('/', (req, res) => {
9    res.send('API is running...')
10 });
11
12 app.get('/api/products', (req, res) => {
13   res.json(products);
14 })
15
16 app.listen(port, () => console.log(`Server running on port ${port}`))
17
```

Créer le Backend et initialiser les routes

- On arrête le serveur
- On re-modifie server.js (page en bas) pour afficher les informations pour un seul produit
- On redémarre le serveur avec npm start
- On ouvre un navigateur et on met l'adresse : localhost:5000/api/products/1
ca va nous afficher les informations textuelles du premier produit

server.js

```
import express from 'express';
import products from './data/products.js';
💡
const port = 5000;

const app = express();

app.get('/', (req, res) => {
  res.send('API is running...')
});

app.get('/api/products', (req, res) => {
  res.json(products);
})

app.get('/api/products/:id', (req, res) => {
  const product = products.find((p) => p._id === req.params.id);
  res.json(product);
})

app.listen(port, () => console.log(`Server running on port ${port}`))
```

Créer le Backend et initialiser les routes

- On arrête le serveur
- Depuis le terminal au niveau du root (nodemon pour pouvoir lancer le serveur backend en arrière plan et le serveur frontend au même temps) :
`npm i -D nodemon concurrently`
- Dans le fichier package.json dans le root, il va afficher les deux dépendances installées
- On modifie package.json dans le root, sous start on ajoute :
`"server": "nodemon backend/server.js"`
- Et depuis la racine dans le terminal on lance : `npm run server`
- A ce stade on n'est plus obligé de lancer le serveur Backend à chaque modification, il marche en continue

Créer le Backend et initialiser les routes

- On arrête le serveur
- Dans package.json dans le root, sous server, ajouter :
"client": "npm start --prefix frontend"
- On teste depuis le terminal niveau root avec : npm run client
ca démarre le serveur Frontend
- On arrête le serveur Frontend
- Dans package.json dans le root, sous client, ajouter :
"dev": "concurrently \"npm run server\" \"npm run client\""

Pour pouvoir lancer les deux serveurs Back et Front au même temps

Créer le Backend et initialiser les routes

- Dans le terminal niveau root, lancer : `npm run dev` et les deux serveurs Back et Front sont lancés et ca nous génère le résultat du front sur le port 3000 dans un navigateur, et si on teste aussi l'adresse du Back sur le port 5000 sur un navigateur ca marche aussi

Créer le Backend et initialiser les routes

- On passe maintenant à la fixation des variables d'environnement
- Maintenant dans le terminal niveau root, arrêter le serveur et lancer :
`npm i -D dotenv` il s'ajoute au fichier `package.json` au root
- Dans le root, créer un fichier `env` le remplir avec :
`NODE_ENV=development`
`PORT=5000`
- Puis modifier le fichier `server.js` comme suit
- Enfin tester depuis le terminal avec `npm run dev` et les deux serveurs back et front vont démarrer

server.js

```
1 import express from 'express';
2 import dotenv from 'dotenv';
3 dotenv.config();
4
5 import products from './data/products.js';
6
7 const port = process.env.PORT || 5000;
8
9 const app = express();
10
11 app.get('/', (req, res) => {
12   res.send('API is running...')
13 });
14
15 app.get('/api/products', (req, res) => {
16   res.json(products);
17 })
18
19 app.get('/api/products/:id', (req, res) => {
20   const product = products.find(p => p._id === req.params.id);
21   res.json(product);
22 })
23
24 app.listen(port, () => console.log(`Server running on port ${port}`))
```

Faire communiquer le Front et le Back

- Maintenant nous allons récupérer les données affichées sur le front depuis le back
- On va aller au frontend et utiliser axios au lieu de `fetch('http://localhost:5000/api/products')` pour récupérer les données depuis le backend
- Dans le terminal, arrêter le serveur, aller au frontend et lancer : `npm i axios`, et dans le `package.json` du frontend axios est ajouté
- Dans `package.json` du frontend on définit un proxy comme suit
- Après depuis le terminal au niveau root, lancer : `npm run dev`

package.json (frontend)

```
{ } package.json X
frontend > { } package.json > ...
1  {
2    "name": "frontend",
3    "version": "0.1.0",
4    "private": true,
5    "proxy": "http://localhost:5000",
6    "dependencies": {
7      "@testing-library/jest-dom": "^5.17.0",
8      "@testing-library/react": "^13.4.0",
9      "@testing-library/user-event": "^13.5.0",
10     "axios": "^1.4.0",
11     "bootstrap": "^5.3.1",
12     "react": "^18.2.0",
13     "react-bootstrap": "^2.8.0",
14     "react-dom": "^18.2.0",
15     "react-icons": "^4.10.1",
16     "react-router-bootstrap": "^0.26.2",
17     "react-router-dom": "^6.14.2",
18     "react-scripts": "5.0.1",
19     "web-vitals": "^2.1.4"
20   },
  Debug
```

Faire communiquer le Front et le Back

- Maintenant on va modifier HomeScreen.js pour qu'il récupère les données depuis le backend
- On va utiliser useState et useEffect
- useState : pour créer un état initial de la variable ou objet demandé et créer une fonction qui met à jour l'état de la variable ou l'objet
- useEffect : mettre à jour un ou plusieurs composants dans le DOM

HomeScreen.js

```
JS HomeScreen.js X
frontend > src > screens > JS HomeScreen.js > [⌕] default
1  import { Row, Col } from 'react-bootstrap'
2  //import products from '../products'
3  import {useEffect, useState} from 'react';
4  import axios from 'axios';
5
6  import Product from '../components/Product'
7
8  const HomeScreen = () => {
9
10     //importer les produits depuis le backend
11     const [products, setProducts] = useState([]);
12     useEffect(() => {
13         const fetchProducts = async () => {
14             //le chemin le prend depuis proxy dans package.json de frontend
15             const { data } = await axios.get('/api/products');
16             setProducts(data);
17         };
18         fetchProducts();
19     }, []);
20
21     return (
22         <>
```

Faire communiquer le Front et le Back

- On enregistre le HomeScreen.js
- Et maintenant les données parviennent au frontend depuis le backend
- Dans le résultat au niveau du navigateur, inspecter, network, on peut voir que products arrivent depuis le backend
- Maintenant on va faire la même modification pour ProductScreen.js, ici la mise à jour dépend de l'id du produit
- Et dans le navigateur ca doit aussi fonctionner pour l'affichage d'un seul produit
- Inspecter, network, montre que le produit provient du backend
- On peut supprimer le fichier products.js dans le frontend

ProductScreen.js

```
2 import { useParams, Link } from 'react-router-dom'
3 import { Row, Col, Image, ListGroup, Card, Button } from 'react-bootstrap'
4 //import products from '../products'
5 import Rating from '../components/Rating';
6
7 import { useState, useEffect } from 'react';
8 import axios from 'axios'
9
10 const ProductScreen = () => {
11
12     const [product, setProduct] = useState({});
13
14     const { id: productId } = useParams();
15     //const product = products.find((p) => p._id === productId)
16
17     useEffect(() => {
18         const fetchProduct = async () => {
19             const { data } = await axios.get(`/api/products/${productId}`);
20             setProduct(data);
21         }
22         fetchProduct();
23     }, [productId]);
24
25     return(
```

Commencer à travailler avec MongoDB

- Faire une inscription dans mongodb.com, en accédant à l'espace créé dans mongodb.com :
 - créer un projet, l'appeler Licorne et créer le projet
 - après cliquer sur Build a Database, choisir l'option gratuite, Provider AWS et créer
 - choisir un login et un mot de passe, et créer l'utilisateur
 - pour l'IP, soit donner une adresse, soit laisser vide pour accéder à la BD depuis n'importe où
 - après une interface déploiement de BD apparaît, on clique sur connect, puis connect your application, on obtient un lien à copier dans le fichier .env, et on ferme

.env

NODE_ENV=development

PORT=5000

MONGO_URI=mongodb+srv://user:password@cluster0.t8mknca.mongodb.net/databasename?retryWrites=true&w=majority

Commencer à travailler avec MongoDB

- Dans l'interface principale de déploiement de BD dans mongodb.com cliquer sur Browse collections et cliquer sur Add my own data
- Nommer la BD : licorne et nommer la collection : products et créer
- La BD et sa table se créent

Commencer à travailler avec MongoDB

- Après, dans le root, créer un fichier example.env

NODE_ENV=development

PORT=5000

MONGO_URI=ADD_YOUR_MONGO_URI

Commencer à travailler avec MongoDB

- Installer MongoDB Compass en local, pour pouvoir gérer la BD depuis l'application el local
- On peut se connecter à une BD en local `mongodb://localhost:27017` comme on peut se connecter à la BD distante en donnant le lien à copier (aller à mongodb.com interface Data services où il y a la BD et sa table (collections) et cliquer sur Overview, puis sur Connect et choisir Connect using MongoDB Compass)

`mongodb+srv://user:password@cluster0.t8mknca.mongodb.net/namedatabase`

- Et connecter soit au BD locale, soit au BD distante
- On peut continuer à fonctionner avec la BD seulement avec l'application MongoDB Compass

Connecter la BD avec l'application

- Pour connecter la BD avec notre application React on va utiliser mongoose
- Dans le terminal au niveau root, arrêter le serveur et lancer `npm i mongoose`
- Puis dans le backend, on crée un dossier config et dans le dossier config on crée un fichier `db.js`
- Et modifier le fichier `server.js`, en haut ajouter :

```
import connectDB from './config/db.js';
```

Puis, avant `const app = express();` ajouter :
`connectDB();`

db.js

```
JS db.js  U X
backend > config > JS db.js > ...
 1  import mongoose from "mongoose";
 2
 3  const connectDB = async () => {
 4      try {
 5          const conn = await mongoose.connect(process.env.MONGO_URI);
 6          console.log(`MongoDB Connected: ${conn.connection.host}`);
 7      } catch (error) {
 8          console.error(`Error: ${error.message}`);
 9          process.exit(1);
10      }
11  }
12
13  export default connectDB
```

Connecter la BD avec l'application

- Après on teste dans le terminal niveau root : `npm run server` et ça nous affiche que le backend est connecté à la BD

Préparation des données de la BD

- Dans le backend, créer un dossier models (appartenant au modèle MVC), et dans models, créer le fichier productModel.js

productModel.js

```
1 import mongoose from "mongoose";
2
3 const reviewSchema = mongoose.Schema({
4   user: {
5     type: mongoose.Schema.Types.ObjectId,
6     required: true,
7     ref: "User",
8   },
9   name: {
10    type: String,
11    required: true,
12  },
13   rating: {
14    type: Number,
15    required: true,
16  },
17   comment: {
18    type: String,
19    required: true,
20  },
21 }, {
22   timestamps: true,
23 });
24
```

productModel.js

```
25 const productSchema = new mongoose.Schema({
26   //utilisateur qui peut acceder aux données de la BD
27   user:{
28     type: mongoose.Schema.Types.ObjectId,
29     required: true,
30     ref: "User",
31   },
32   //données de la BD
33   name: {
34     type: String,
35     required: true,
36   },
37   image:{
38     type: String,
39     required: true,
40   },
41   brand: {
42     type: String,
43     required : true,
44   },
45   category: {
46     type: String,
47     required : true,
48   },
49 }
```

productModel.js

```
49     description: {  
50       type: String,  
51       required : true,  
52     },  
53     reviews: [reviewSchema],  
54     rating:{  
55       type: Number,  
56       required: true,  
57       default: 0,  
58     },  
59     numReviews:{  
60       type: Number,  
61       required: true,  
62       default: 0,  
63     },  
64     price:{  
65       type: Number,  
66       required: true,  
67       default: 0,  
68     },
```

productModel.js

```
69     countInStock:{
70         type: Number,
71         required: true,
72         default: 0,
73     },
74 }, {
75     timestamps: true,
76 });
77
78 const Product = mongoose.model("Product", productSchema);
79
80 export default Product;
```

Préparation des données de la BD

- Dans le backend, dans models, créer le fichier userModel.js

userModel.js

```
1 import mongoose from "mongoose";
2
3 const userSchema = mongoose.Schema({
4   name: {
5     type: String,
6     required: true,
7   },
8   email: {
9     type: String,
10    required: true,
11    unique: true,
12  },
13  password: {
14    type: String,
15    required: true,
16  },
17  isAdmin: {
18    type: Boolean,
19    required: true,
20    default: false,
21  },
22 }, {
23   timestamps: true,
24 });
```

userModel.js

```
23     }, {  
24         timestamps: true,  
25     });  
26  
27     const User = mongoose.model('User', userSchema);  
28  
29     export default User;
```

Préparation des données de la BD

- Dans le backend, dans models, créer le fichier orderModel.js

orderModel.js

```
1 import mongoose from 'mongoose'
2
3 const orderSchema = mongoose.Schema({
4   user : {
5     type:mongoose.Schema.Types.ObjectId,
6     required:true,
7     ref: 'User'
8   },
9   orderItems : [
10     {
11       name : { type : String, required: true },
12       qty : { type : Number, required: true },
13       image: { type : String, required: true },
14       price : { type : Number, required: true },
15       product : { type : mongoose.Schema.Types.ObjectId,
16         required: true,
17         ref: 'Product'
18       },
19     }
20   ],
21   shippingAddress : {
22     address : { type: String , required: true },
23     city : { type: String , required: true },
24     postalCode : { type: String , required: true },
```

orderModel.js

```
25     country : { type: String , required: true },
26   },
27   paymentMethod : {
28     type:String,
29     required:true
30   },
31   paymentResult : {
32     id: {type: String} ,
33     status: {type: String} ,
34     update_time: {type: String} ,
35     email_address: {type: String} ,
36   },
37   itemsPrice : {
38     type:Number,
39     required:true,
40     default: 0.0
41   },
42   taxPrice : {
43     type:Number,
44     required:true,
45     default: 0.0
46   },
47   shippingPrice : {
48     type:Number,
```

orderModel.js

```
49         required:true,  
50         default: 0.0  
51     },  
52     totalPrice : {  
53         type:Number,  
54         required:true,  
55         default: 0.0  
56     },  
57     isPaid : {  
58         type:Boolean,  
59         required:true,  
60         default: false  
61     },  
62     paidAt: {  
63         type: Date  
64     },  
65     isDelivered : {  
66         type:Boolean,  
67         required:true,  
68         default: false  
69     },  
70     deliveredAt: {  
71         type: Date  
72     },
```

orderModel.js

```
73  
74   },  
75   {  
76     timestamps:true  
77   });  
78  
79   const Order = mongoose.model('Order', orderSchema)  
80  
81   export default Order
```

Préparation des données de la BD

- On va maintenant injecter des données dans la BD
- Dans le fichier products.js qui est dans backend/data, supprimer les ID
- Arrêter le serveur
- Lancer dans le terminal (niveau root) : `npm run pbaryleptjs`
cryptage des données importantes comme le mot de passe
- Dans backend/data, créer un fichier users.js

users.js

```
backend > data > JS users.js > [🔗] default
1  import bcrypt from 'bcryptjs'
2
3  const users = [
4    {
5      name: 'Test',
6      email: 'test@la.store',
7      password : bcrypt.hashSync('123456',10),
8      isAdmin : true
9    },
10   {
11     name: 'User',
12     email: 'user@la.store',
13     password : bcrypt.hashSync('123',10),
14     isAdmin : false
15   }
16 ];
17
18 export default users
```

Préparation des données de la BD

- On va installer un package supplémentaire
- Arrêter le serveur, niveau root, lancer : `npm i colors`
- Dans le backend, créer le fichier `seeder.js` pour injecter des données dans la BD

seeder.js

```
1 import mongoose from 'mongoose'
2 import dotenv from 'dotenv'
3 import colors from 'colors'
4 import users from './data/users.js'
5 import products from './data/products.js'
6 import User from './models/userModel.js'
7 import Product from './models/productModel.js'
8 import Order from './models/orderModel.js'
9 import connectDB from './config/db.js'
10
11 dotenv.config();
12
13 await connectDB();
14
15 const importData = async () => {
16   try {
17     await Order.deleteMany()
18     await Product.deleteMany()
19     await User.deleteMany()
20
21     const createdUsers = await User.insertMany(users)
22
23     const adminUser = createdUsers[0]._id
```

seeder.js

```
24
25     const sampleProducts = products.map((product) => {
26       return { ...product, user: adminUser }
27     })
28
29     await Product.insertMany(sampleProducts)
30
31     console.log('Data Imported!'.green.inverse)
32     process.exit()
33   } catch (error) {
34     console.error(`${error}`.red.inverse)
35     process.exit(1)
36   }
37 }
38
39 const destroyData = async () => {
40   try {
41     await Order.deleteMany()
42     await Product.deleteMany()
43     await User.deleteMany()
44
45     console.log('Data Destroyed!'.red.inverse)
46     process.exit()
47   } catch (error) {
```

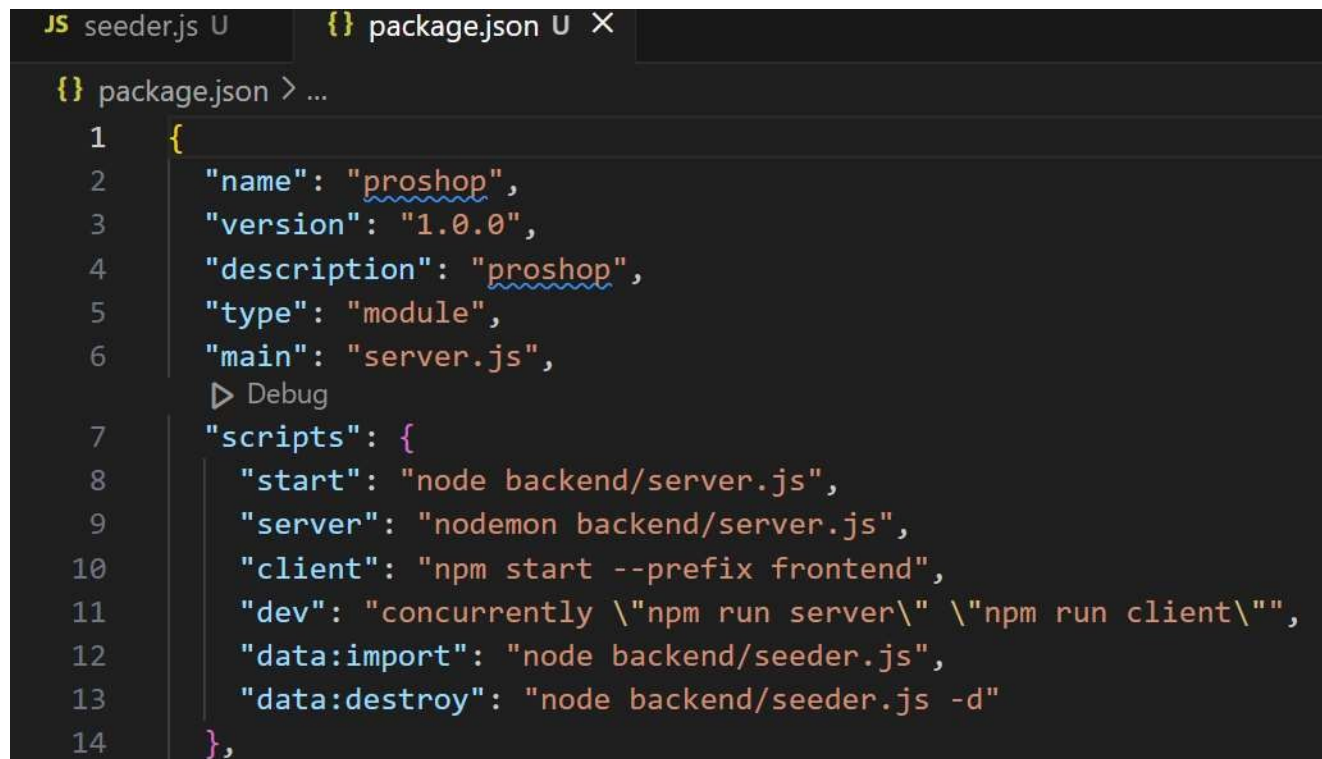
seeder.js

```
48     console.error(`${error}`.red.inverse)  
49     process.exit(1)  
50 }  
51 }  
52  
53 if (process.argv[2] === '-d') {  
54     destroyData();  
55 } else {  
56     importData();  
57 }
```

Préparation des données de la BD

- Après aller au package.json dans le root, et en bas de dev, faire des ajouts des lignes d'importation et destruction de données

package.json (niveau root)



The screenshot shows a code editor with two tabs: 'seeder.js U' and 'package.json U X'. The 'package.json' tab is active, displaying the following JSON content:

```
1  {
2    "name": "proshop",
3    "version": "1.0.0",
4    "description": "proshop",
5    "type": "module",
6    "main": "server.js",
7    "scripts": {
8      "start": "node backend/server.js",
9      "server": "nodemon backend/server.js",
10     "client": "npm start --prefix frontend",
11     "dev": "concurrently \"npm run server\" \"npm run client\"",
12     "data:import": "node backend/seeder.js",
13     "data:destroy": "node backend/seeder.js -d"
14   },
```

Préparation des données de la BD

- Après, pour détruire les données dans la BD faire dans le terminal, niveau root : `npm run data:destroy` (ca génère un message de confirmation en rouge)
- Pour importer les données dans la BD faire dans le terminal, niveau root : `npm run data:import` (ca génère un message de confirmation en vert)
- On peut vérifier les données de la BD, et on va trouver que les produits et les utilisateurs ont été ajoutés

Utilisation de Postman pour le Backend

- Installer et ouvrir Postman
- Il faut que le site soit en exécution (npm run dev)
- Créer un nouveau workspace, l'appeler Licorne
- Aller dans Environments et créer un environnement appelé Licorne environment et ajouter la variable : baseUrl
initial value : <http://localhost:50000/api>
current value : <http://localhost:50000/api>

Postman

Home

Workspaces

API Network

Explore

Search Postman

Invite

Upgrade

Licorne

New

Import

Overview

POST Register User

Licorne Environnement

Licorne Environnement

Collections

APIs

Environments

History

Globals

Licorne Environnement

Licorne Environnement

Fork 0

Save

Share

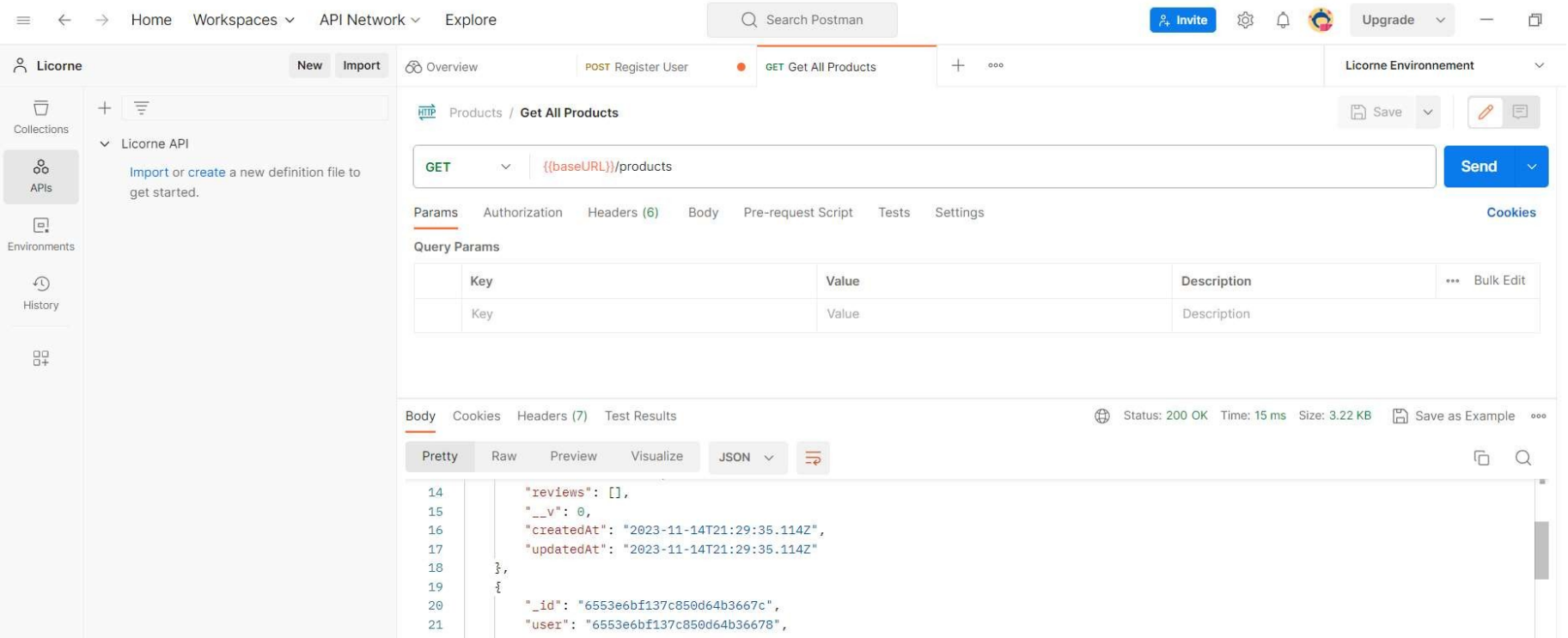
Filter variables

	Variable	Type	Initial value	Current value	
☒	baseUrl	default	http://localhost:5000/api	http://localhost:5000/api	
	Add new variable				

Utilisation de Postman pour le Backend

- Aller à APIs, créer une API à appeler Licorne API
- Après dans Collections, créer la collection : Products
- Clic droit sur Products et Add request (créer requête comme suit) et cliquer sur send pour tester les requêtes, et save à la fin

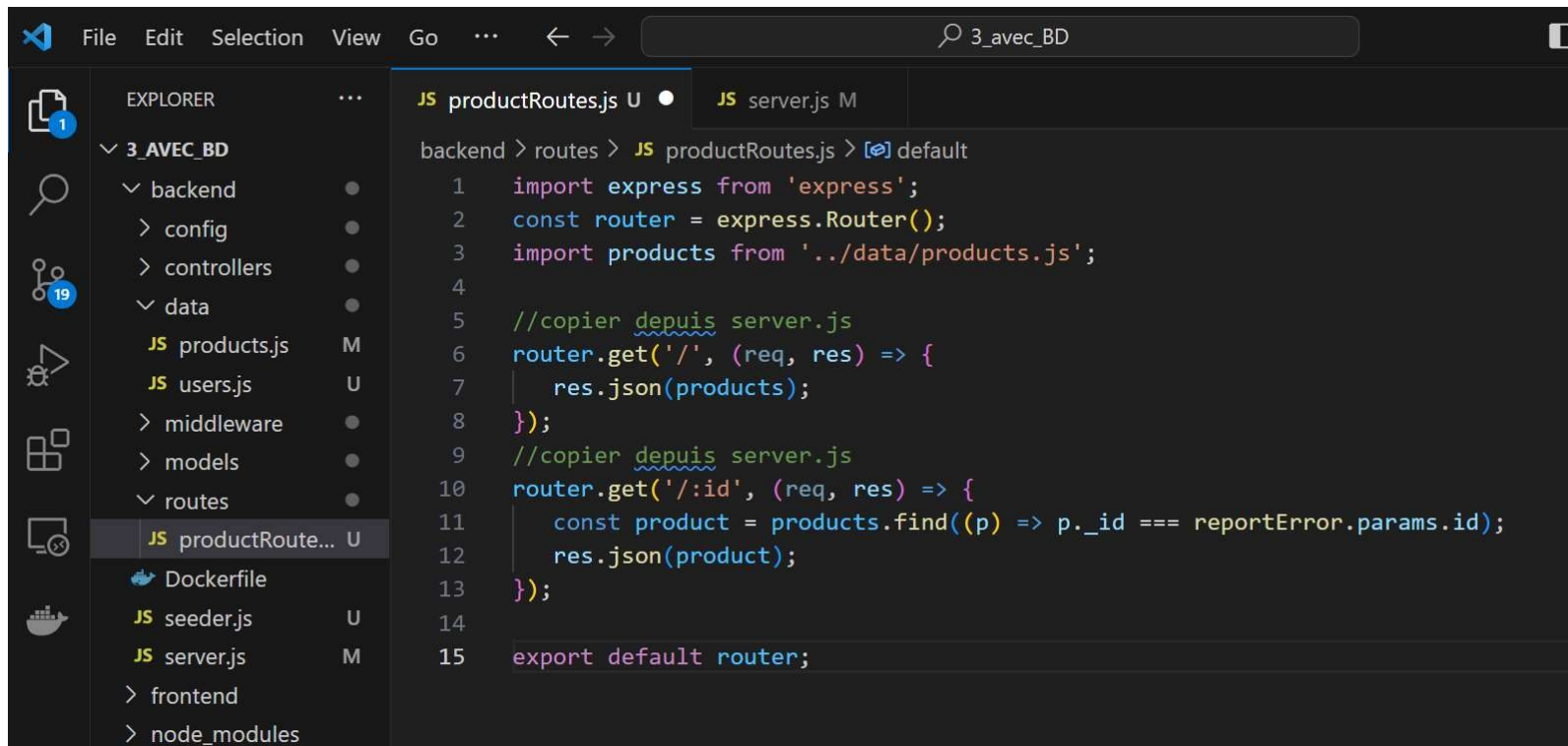
Postman



Utilisation de Postman pour le Backend

- Maintenant on va créer les routes pour le Backend, dans le Backend créer un dossier routes, et dans routes, créer un fichier productRoutes.js (voir en bas)
- Dans productRoutes.js, on va couper/coller deux instructions depuis server.js (voir en bas) et couper/coller aussi l'importation de products (voir en bas) pour tester l'importation des produits depuis le fichier
- Dans server.js, ajouter en haut :
`import productRoutes from './routes/productRoutes.js';`
- Et toujours dans server.js, ajouter en bas (après API is running...) :
`app.use('/api/products', productRoutes);`
- Et tester

productRoutes.js



The screenshot shows the Visual Studio Code editor interface. The Explorer sidebar on the left displays a project structure with a folder named '3_AVEC_BD' containing subfolders 'backend', 'data', 'middleware', 'models', and 'routes'. The 'routes' folder is expanded, showing 'productRoutes.js' as the selected file. The main editor area displays the code for 'productRoutes.js', which includes imports for 'express' and 'products', and two GET routes: one for the root path '/' returning all products, and another for '/:id' returning a specific product by ID. The code is as follows:

```
backend > routes > JS productRoutes.js > [default]
1  import express from 'express';
2  const router = express.Router();
3  import products from '../data/products.js';
4
5  //copier depuis server.js
6  router.get('/', (req, res) => {
7    res.json(products);
8  });
9  //copier depuis server.js
10 router.get('/:id', (req, res) => {
11   const product = products.find((p) => p._id === req.query.id);
12   res.json(product);
13 });
14
15 export default router;
```

Utilisation de Postman pour le Backend

- Maintenant on va faire l'importation des produits depuis la BD
- Dans le backend, on va créer un dossier middleware, et dans middleware créer le fichier `asyncHandler.js` (voir en bas)
- On modifie `productRoutes.js` (voir en bas)
- Puis on teste, et les données sont récupérées depuis la BD

asyncHandler.js

```
JS productRoutes.js U ● JS asyncHandler.js U X JS server.js M
backend > middleware > JS asyncHandler.js > [🔗] default
1  const asyncHandler = fn => (req, res, next) => {
2    Promise.resolve(fn(req, res, next)).catch(next);
3  }
4
5  export default asyncHandler;
```

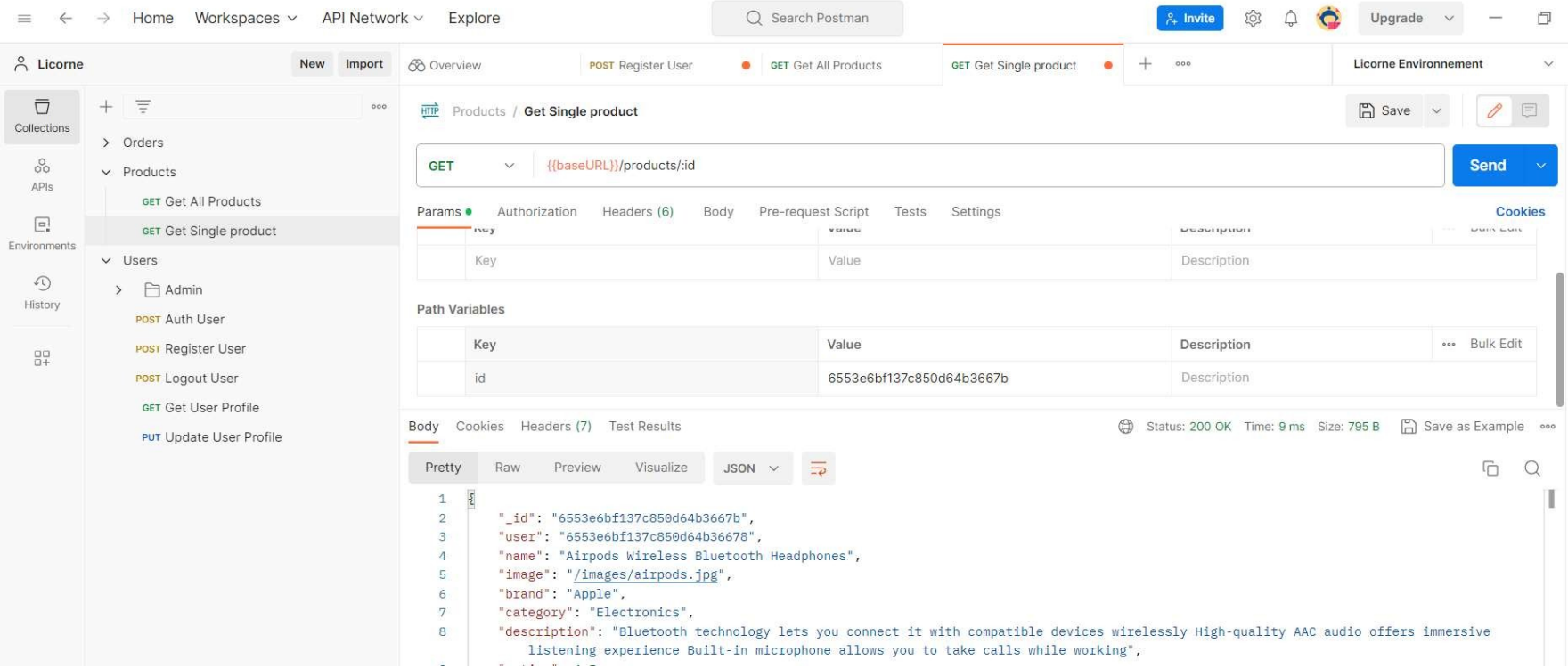
productRoutes.js

```
JS server.js M JS productRoutes.js U ●
backend > routes > JS productRoutes.js > [?] default
1  import express from 'express';
2  const router = express.Router();
3  import asyncHandler from '../middleware/asyncHandler.js';
4  import Product from '../models/productModel.js';
5
6  router.get('/', asyncHandler(async (req, res) => {
7    const products = await Product.find({});
8    res.json(products);
9  }));
10
11 router.get('/:id', asyncHandler(async (req, res) => {
12   const product = await Product.findById(req.params.id);
13
14   if(product){
15     return res.json(product);
16   }
17   res.status(404).json({ message: 'Product not found' });
18 }));
19
20 export default router;
```

Utilisation de Postman pour le Backend

- Dans Postman, clic droit sur Products, et créer une nouvelle requête
- Send avec le bon ID, et enregistrer

Postman



Utilisation de Postman pour le Backend

- Maintenant on va créer une partie qui gère les erreurs
- Dans backend, middleware, créer un fichier `errorMiddleware.js`
- Dans `server.js`, en haut ajouter : `import { notFound, errorHandler } from './middleware/errorMiddleware.js'`
et après `app.use('/api/products', productRoutes);`
ajouter :
`app.use(notFound);`
`app.use(errorHandler);`
- Après, modifier `productRoutes.js`
- Enfin, en testant sur Postman, il va gérer les erreurs (cas d'un faux id)

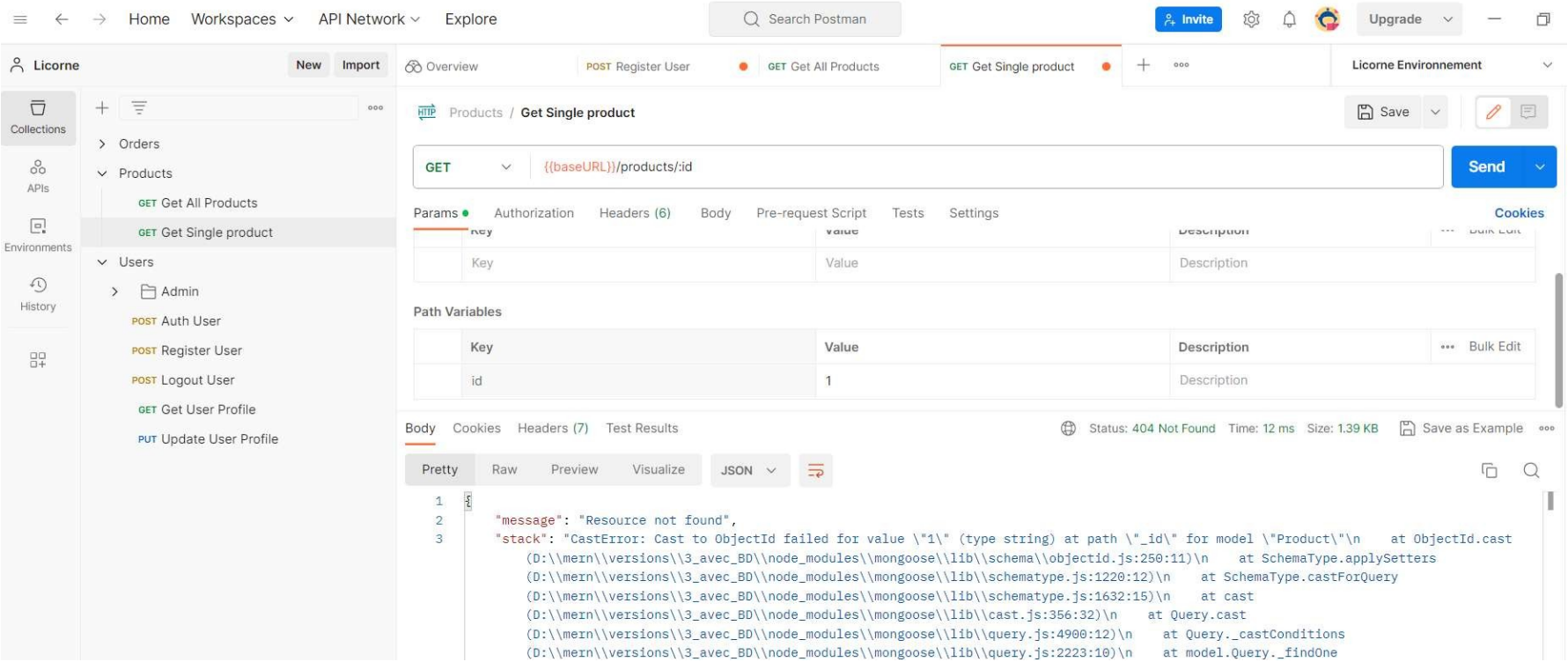
errorMiddleware.js

```
1  const notFound =(req, res , next)=> {
2    const error = new Error(`Not Found - ${req.originalUrl}`)
3    res.status(404)
4    next(error)
5  }
6
7  const errorHandler = (err , req ,res , next) => {
8    let statusCode = res.statusCode === 200 ? 500 : res.statusCode
9    let message = err.message
10
11    if(err.name === 'CastError' && err.kind === 'ObjectId'){
12      message = `Resource not found`
13      statusCode = 404
14    }
15
16    res.status(statusCode).json({
17      message,
18      stack: process.env.NODE_ENV === 'production' ? '0' : err.stack,
19    });
20  };
21
22  export { notFound,errorHandler }
```

productRoutes.js

```
20 router.get('/:id', asyncHandler(async (req, res) => {
21     const product = await Product.findById(req.params.id);
22
23     if(product){
24         return res.json(product);
25     }
26     else{
27         res.status(404);
28         throw new Error('Resource not found');
29     }
30 }));
31
32
33 export default router;
```

Postman



Utilisation de Postman pour le Backend

- Maintenant on va passer à la création du controleur produit dans le backend
- On va créer un controleur pour chaque route
- Dans le backend créer un dossier controllers, puis dans controllers, créer le fichier productController.js
- Et modifier le fichier productRoutes.js
- Et tester le site et dans Postman

productController.js

```
1  import asyncHandler from '../middleware/asyncHandler.js';
2  import Product from '../models/productModel.js';
3
4  // @desc fetch all products
5  // @route GET /api/products
6  // @access Public
7  const getProducts = asyncHandler(async (req, res) => {
8    const products = await Product.find();
9    res.json(products);
10 });
11 // @desc fetch a product
12 // @route GET /api/products/:id
13 // @access Public
14 const getProductById = asyncHandler (async (req, res) => {
15   const product = await Product.findById(req.params.id);
16   if (product) {
17     return res.json(product);
18   } else {
19     res.status(404);
20     throw new Error('Resource not found');
21   }
22 });
23 export { getProducts, getProductById };
```

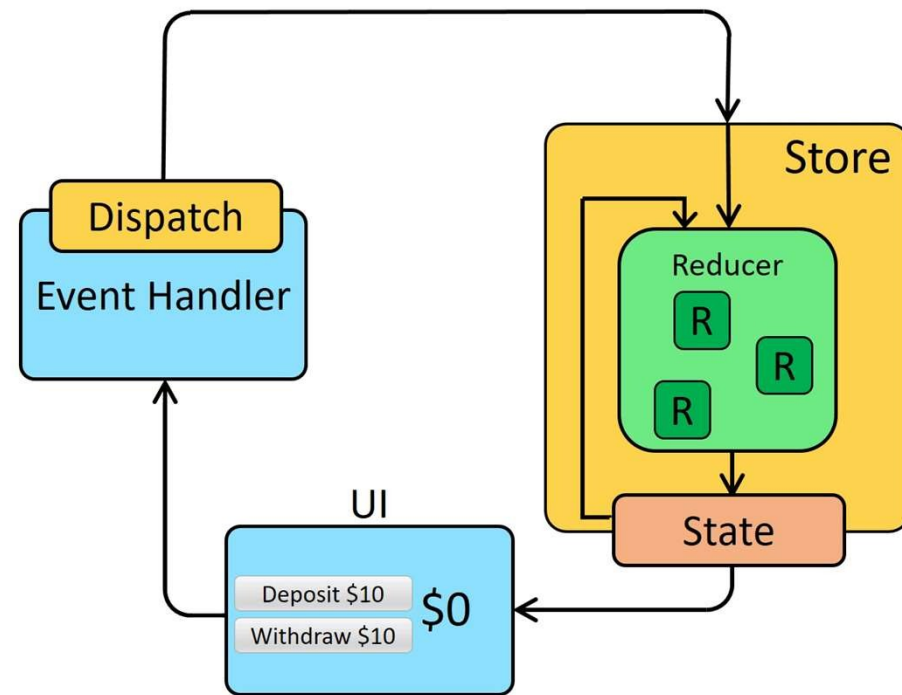
productRoutes.js

```
JS server.js M    JS productController.js U ●    JS productRoutes.js U ●
backend > routes > JS productRoutes.js > ...
1  import express from 'express';
2  const router = express.Router();
3
4  import { getProducts, getProductById } from '../controllers/productController.js';
5
6  router.route('/').get(getProducts);
7  router.route('/:id').get(getProductById);
8
9  export default router;
```

Utilisation de Redux

- Un framework populaire qui permet de gérer les états (states)
- On va l'utiliser avec React
- Il permet de centraliser et d'uniformiser la gestion des états d'une application

Redux



Utilisation de Redux

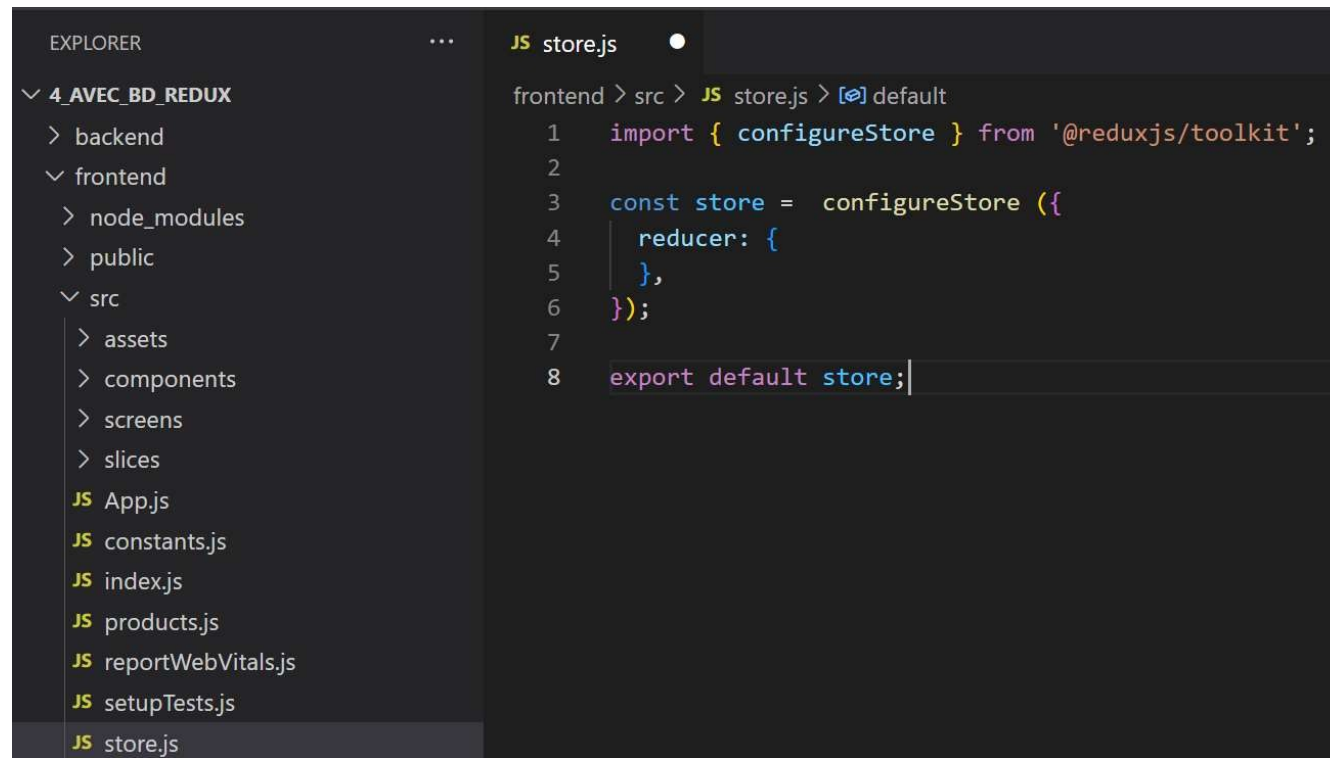
- Dans le terminal on va installer le package de Redux (niveau frontend) :
`npm i @reduxjs/toolkit react-redux`
- Dans le package.json du frontend, le Redux est ajouté
- Dans le frontend/src, on crée un fichier store.js
- Et modifier index.js :
 - d'abord importer en haut ces 2 bibliothèques :

```
import { Provider } from 'react-redux';
```

```
import store from './store';
```

- après utiliser le store comme props dans le fichier index.js (voir page suivante)

store.js



The image shows a code editor interface. On the left is the 'EXPLORER' sidebar with a file tree. The tree is expanded to show the 'src' directory, which contains several files: 'App.js', 'constants.js', 'index.js', 'products.js', 'reportWebVitals.js', 'setupTests.js', and 'store.js'. The 'store.js' file is selected and highlighted. The main editor area on the right shows the code for 'store.js'. The code imports 'configureStore' from '@reduxjs/toolkit', creates a 'store' constant with a 'reducer' object, and exports the 'store' as the default export.

```
EXPLORER
  4_AVEC_BD_REDUX
    > backend
    > frontend
      > node_modules
      > public
      > src
        > assets
        > components
        > screens
        > slices
        JS App.js
        JS constants.js
        JS index.js
        JS products.js
        JS reportWebVitals.js
        JS setupTests.js
        JS store.js

JS store.js
frontend > src > JS store.js > [🔗] default
1  import { configureStore } from '@reduxjs/toolkit';
2
3  const store = configureStore ({
4    reducer: {
5    },
6  });
7
8  export default store;
```

index.js

```
JS index.js  ×
frontend > src > JS index.js > ...
28  const root = ReactDOM.createRoot(document.getElementById('root'));
29  root.render(
30    <React.StrictMode>
31      <Provider store={store}>
32        <RouterProvider router={router} />
33      </Provider>
34    </React.StrictMode>
35  );
36
37  reportWebVitals();
38
```

Utilisation de Redux

- Après, pour vérifier que l'extension de Redux marche, il faut inspecter la page web dans Chrome et on trouvera l'extension Redux qui doit marcher

Utilisation de Redux

- Maintenant, on va créer des constantes qu'on va utiliser
- Créer un fichier constants.js dans frontend/src

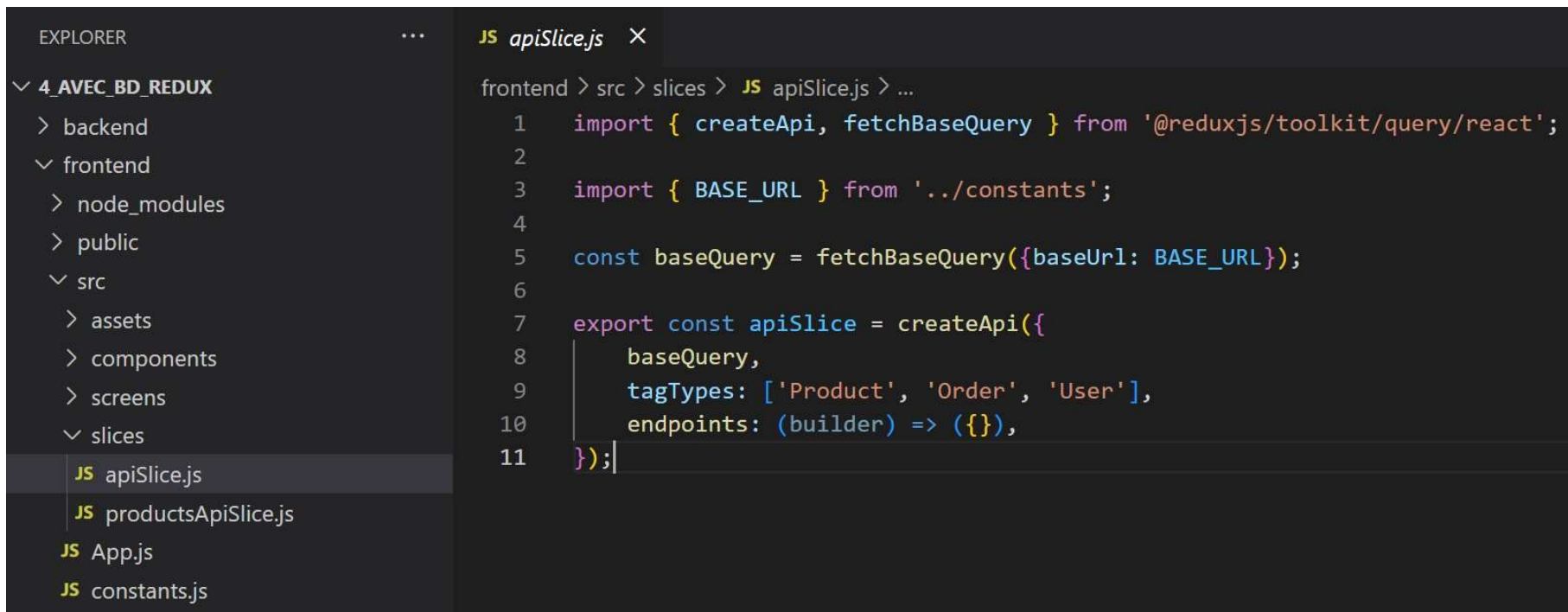
constants.js

```
JS constants.js ●  
frontend > src > JS constants.js > ...  
1 |  
2 export const BASE_URL = '';  
3  
4 export const PRODUCTS_URL = '/api/products';  
5 export const USERS_URL = '/api/users';  
6 export const ORDERS_URL = '/api/orders';  
7 export const PAYPAL_URL = '/api/config/paypal';  
8
```

Utilisation de Redux

- Maintenant, on va suivre le principe de Redux en créant des morceaux (Slices) et chaque morceau aura un role (exemple : usersSlice, productsSlice, ordersSlice...)
- Dans frontend/src créer un dossier slices
- Dans le dossier slices, créer apiSlice.js qui sera le parent des autres slices liés à la BD (products, users, orders)
- Et modifier le fichier store.js

apiSlice.js



The image shows a code editor interface. On the left is the 'EXPLORER' sidebar with a file tree. The tree is expanded to '4_AVEC_BD_REDUX' > 'src' > 'slices', where 'apiSlice.js' is selected. Other files in the 'slices' folder are 'productsApiSlice.js', 'App.js', and 'constants.js'. The main editor area shows the code for 'apiSlice.js' with line numbers 1 through 11. The code imports 'createApi' and 'fetchBaseQuery' from '@reduxjs/toolkit/query/react', imports 'BASE_URL' from '../constants', defines a 'baseQuery' constant, and exports a 'createApi' configuration object with 'baseQuery', 'tagTypes' (['Product', 'Order', 'User']), and 'endpoints' (a builder function returning an empty object).

```
EXPLORER
4_AVEC_BD_REDUX
  backend
  frontend
    node_modules
    public
    src
      assets
      components
      screens
      slices
        JS apiSlice.js
        JS productsApiSlice.js
      JS App.js
      JS constants.js

JS apiSlice.js
frontend > src > slices > JS apiSlice.js > ...
1  import { createApi, fetchBaseQuery } from '@reduxjs/toolkit/query/react';
2
3  import { BASE_URL } from '../constants';
4
5  const baseQuery = fetchBaseQuery({baseUrl: BASE_URL});
6
7  export const apiSlice = createApi({
8    baseQuery,
9    tagTypes: ['Product', 'Order', 'User'],
10   endpoints: (builder) => ({}),
11 });
```

store.js

JS store.js

frontend > src > JS store.js > [🔗] default

```
1  import { configureStore } from '@reduxjs/toolkit';
2  import { apiSlice } from '../slices/apiSlice';
3
4  const store = configureStore ({
5    reducer: {
6      [apiSlice.reducerPath]: apiSlice.reducer,
7    },
8    middleware: (getDefaultMiddleware) => getDefaultMiddleware().concat
9      (apiSlice.middleware),
10    devTools: true,
11  });
12
13  export default store;
```

Utilisation de Redux

- On teste l'application avec `npm run dev`
- Normalement elle marche correctement, et dans `inspecter/Redux` on doit voir apparaître le state api

Utilisation de Redux

- Maintenant, nous allons créer le premier slice enfant pour la recherche des produits depuis la BD à travers le backend
- Dans le dossier frontend/slices créer productsApiSlice.js
- Après aller modifier HomeScreen.js (nous n'allons plus utiliser des requetes avec axios, ca sera géré avec Redux)
- Mais avant de tester l'affichage des produits, et l'importation des données qui va marcher avec Redux, il faut finaliser le Loader (chargement) et Le Message (messages)

Utilisation de Redux

- Donc on va créer d'abord un loader pour le chargement
- Dans frontend/components, créer Loader.js
- Après modifier HomeScreen.js (voir fichier en bas) :
 - en haut :

`import Loader from '../components/Loader'`

- modifier `<h2>Loading...</h2>` par `<Loader />` Après faire la même chose avec le fichier ProductScreen.js (voir fichier en bas)

Loader.js

```
JS Loader.js •
frontend > src > components > JS Loader.js > ...
1  import React from 'react'
2  import {Spinner} from 'react-bootstrap'
3
4  const Loader = () => {
5    return (
6      <Spinner animation='border' role='status' style= {{ width:'100px' , height: '100px',
7        margin : 'auto' , display:'block'    }} >
8        <span className='sr-only'> Loading ... </span>
9      </Spinner>
10    )
11  }
12
13  export default Loader
```

Utilisation de Redux

- Maintenant, on va créer la même chose que Loader mais pour les messages
- Dans components, créer Message.js
- Après modifier HomeScreen.js (voir fichier en bas) :
 - en haut :

```
import Message from '../components/Message'
```

- modifier :

```
<div>{ error?.data?.message || error.error } </div>
```

par :

```
<Message variant='danger'>{ error?.data?.message || error.error } </Message>
```

- Après faire la même chose avec le fichier ProductScreen.js (voir fichier en bas)

Message.js

```
JS Message.js X
frontend > src > components > JS Message.js > ...
1  import React from 'react'
2  import { Alert } from 'react-bootstrap'
3
4  const Message = ({ variant , children }) => {
5    return (
6      <Alert variant={variant}>{children}</Alert>
7    )
8  }
9
10 Message.defaultProps = {
11   variant: 'info' ,
12 }
13
14 export default Message
```

HomeScreen.js

```
JS apiSlice.js U JS HomeScreen.js U X JS ProductScreen.js U JS productsApiSlice.js U
frontend > src > screens > JS HomeScreen.js > ...
1 import { Row, Col } from 'react-bootstrap'
2
3 import Loader from '../components/Loader'
4 import Message from '../components/Message'
5
6 //import products from '../products'
7 //import {useEffect, useState} from 'react';
8 //import axios from 'axios';
9 import { useGetProductsQuery } from '../slices/productsApiSlice';
10
11 import Product from '../components/Product'
12
13 const HomeScreen = () => {
14
15   const { data: products, isLoading, error } = useGetProductsQuery();
16   //importer les produits depuis le backend
17   //const [products, setProducts] = useState([]);
18   //useEffect(() => {
19     //const fetchProducts = async () => {
20       //le chemin le prend depuis proxy dans package.json de frontend
21       //const { data } = await axios.get('/api/products');
```

HomeScreen.js

```
frontend > src > screens > JS HomeScreen.js > ...
22 | //setProducts(data);
23 | //};
24 | //fetchProducts();
25 | //}, []));
26 |
27 | return (
28 |   <>
29 |     {isLoading ? (
30 |       <Loader />
31 |     ): error ? (<Message variant='danger'>{ error?.data?.message || error.error } </Message>) :
32 |     (<>
33 |       <h1>Latest Products</h1>
34 |       <Row>
35 |         {products.map((product) => (
36 |           <Col key={product._id} sm={12} md={6} lg={4} xl={3}>
37 |             <Product product={product} />
38 |           </Col>
39 |         ))}
40 |       </Row>
41 |     </>) }
```

HomeScreen.js

```
42     </>  
43   );  
44 };  
45  
46 export default HomeScreen
```

Utilisation de Redux

- Maintenant, nous allons nous occuper de l'importation d'un seul produit avec Redux
- D'abord modifier productsApiSlice.js
- Après modifier ProductScreen.js (nous n'allons plus utiliser des requetes avec axios, ca sera géré avec Redux)
- Après tester l'importation de plusieurs ou d'un seul produit (ca devrait marcher avec Redux)

productsApiSlice.js

frontend > src > slices > JS apiSlice.js > productsApiSlice > endpoints > getProducts

```
1  import { PRODUCTS_URL } from "../constants";
2  import { apiSlice } from "../apiSlice";
3
4  export const productsApiSlice = apiSlice.injectEndpoints({
5    endpoints: (builder) => ({
6
7      getProducts: builder.query({
8        query: () => ({
9          url: PRODUCTS_URL,
10         }),
11        //providesTags: ['Product'],
12        //5 seconds
13        keepUnusedDataFor: 5,
14      }),
15
16  })
```

productsApiSlice.js

```
frontend > src > slices > JS apiSlice.js > [🔗] productsApiSlice > 📦 endpoints > 🔑 getProducts
16
17     getProductDetails: builder.query({
18       query: (productId) => ({
19         url: `${PRODUCTS_URL}/${productId}`,
20       }),
21       keepUnusedDataFor: 5,
22     }),
23
24   }),
25 });
26
27 export const { useGetProductsQuery, useGetProductDetailsQuery } = productsApiSlice;
28
```

ProductScreen.js

```
frontend > src > screens > JS ProductScreen.js > ...
1
2 import { useParams, Link } from 'react-router-dom'
3 import { Row, Col, Image, ListGroup, Card, Button } from 'react-bootstrap'
4 //import products from '../products'
5 import Rating from '../components/Rating';
6 import Loader from '../components/Loader';
7 import Message from '../components/Message';
8 import { useGetProductDetailsQuery } from '../slices/productsApiSlice';
9
10 //import { useState, useEffect } from 'react';
11 //import axios from 'axios'
12
13 const ProductScreen = () => {
14
15     //const [product, setProduct] = useState({});
16
17     const { id: productId } = useParams();
18     //const product = products.find((p) => p._id === productId)
19 }
```

ProductScreen.js

```
frontend > src > screens > JS ProductScreen.js > ...
20  /*
21  useEffect(() => {
22      const fetchProduct = async () => {
23          const { data } = await axios.get(`/api/products/${productId}`);
24          setProduct(data);
25      }
26      fetchProduct();
27  }, [productId]);
28  */
29
30  const { data: product, isLoading, error } = useGetProductDetailsQuery
31  (productId);
32
33
34  return(
35      <>
36          <Link className='btn btn-light my-3' to='/'>
37              | Go Back
38          </Link>
39  )
```

ProductScreen.js

```
frontend > src > screens > JS ProductScreen.js > ...
40 { isLoading ? (
41   <Loader />
42 ) : error ? (
43   <Message variant='danger'>{ error?.data?.message || error.error } </Message>
44 ) : (<Row>
45   <Col md={5}>
46     <Image src={product.image} alt={product.name} fluid />
47   </Col>
48   <Col md={4}>
49     <ListGroup variant='flush'>
50       <ListGroup.Item>
51         <h6>{product.name}</h6>
52       </ListGroup.Item>
53       <ListGroup.Item>
54         <Rating value={product.rating} text={` ${product.numReviews} reviews`} />
55       </ListGroup.Item>
56       <ListGroup.Item>
57         Price: ${product.price}
58       </ListGroup.Item>
59       <ListGroup.Item>
60         Description: {product.description}
```

ProductScreen.js

```
frontend > src > screens > JS ProductScreen.js > ...
61     </ListGroup.Item>
62   </ListGroup>
63 </Col>
64 <Col md={3}>
65   <Card>
66     <ListGroup variant='flush'>
67       <ListGroup.Item>
68         <Row>
69           <Col>Price:</Col>
70           <Col>
71             <strong>${product.price}</strong>
72           </Col>
73         </Row>
74       </ListGroup.Item>
75       <ListGroup.Item>
76         <Row>
77           <Col>Status:</Col>
78           <Col>
79             <strong>{product.countInStock > 0 ? 'In Stock' :
80               'Out of Stock'}</strong>
81           </Col>
```

ProductScreen.js

```
frontend > src > screens > JS ProductScreen.js > ...
82      </Row>
83    </ListGroup.Item>
84    <ListGroup.Item>
85      <Button
86        className='btn-block'
87        type='button'
88        disabled={product.countInStock === 0}
89      >
90        Add To cart
91      </Button>
92    </ListGroup.Item>
93  </ListGroup>
94 </Card>
95 </Col>
96 </Row> ) }
97 </>
98 );
99 };
100
101 export default ProductScreen
```

Fin du cours

Merci pour votre attention

Dr. Tarek BCHINI